Hot200

1. 滑动窗口

LeetCode239. 滑动窗口最大值

> 给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。
>
> 返回 *滑动窗口中的最大值*。

示例 1：

输入： nums = [1,3,-1,-3,5,3,6,7], k = 3
输出： [3,3,5,5,6,7]
解释：

滑动窗口的位置	最大值
[1 3 -1] -3 5 3 6 7	**3**
1 [3 -1 -3] 5 3 6 7	**3**
1 3 [-1 -3 5] 3 6 7	**5**
1 3 -1 [-3 5 3] 6 7	**5**
1 3 -1 -3 [5 3 6] 7	**6**
1 3 -1 -3 5 [3 6 7]	**7**

思路解析：双端队列维护单调递减

代码块

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] res = new int[n-k+1];
        // 双端队列中存储数组下标
        Deque<Integer> queue = new LinkedList<>();
        for(int i=0; i<n; i++){
            // 移除队尾比当前小的元素
            while(!queue.isEmpty() && nums[i]>=nums[queue.peekLast()]){
                queue.pollLast();
            }
            queue.offer(i); // 存储下标
            // 移除超出窗口左边界的队首
            if(queue.peekFirst() <= i-k){
```

```
                queue.pollFirst();
            }
            // 窗口形成，记录最大值
            if(i>=k-1){
                res[i-k+1] = nums[queue.peekFirst()];
            }
        }
        return res;
    }
}
```

LeetCode3. 无重复字符的最长子串【华为】

> 给定一个字符串 `s` ，请你找出其中不含有重复字符的 **最长** 子串 的长度。

示例 1:

输入: s = "abcabcbb"
输出: 3
解释: 因为无重复字符的最长子串是 "abc"，所以其长度为 3。注意 "bca" 和 "cab" 也是正确答案。

代码块

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int maxLen = 0;
        HashMap<Character, Integer> map = new HashMap<>();
        int left = 0;
        for(int i=0; i<s.length(); i++){
            char c = s.charAt(i);
            if(map.containsKey(c)){
                left = Math.max(left,map.get(c)+1);
            }
            map.put(c,i);
            maxLen = Math.max(maxLen,i-left+1);
        }
        return maxLen;
    }
}
```

LeetCode438. 找到字符串中所有字母异位词

> 给定两个字符串 `s` 和 `p`，找到 `s` 中所有 `p` 的 异位词 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

示例 1:

输入: s = "cbaebabacd", p = "abc"
输出: [0,6]
解释:
起始索引等于 0 的子串是 "cba", 它是 "abc" 的异位词。
起始索引等于 6 的子串是 "bac", 它是 "abc" 的异位词。

代码块

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> res = new ArrayList<>();
        int[] need = new int[26];
        int[] window = new int[26];
        for(int i=0; i<p.length(); i++) need[p.charAt(i)-'a']++;
        int right = 0;
        int left = 0;
        while(right < s.length()){
            char c = s.charAt(right);
            right++;
            window[c-'a']++;
            while(right-left > p.length()){
                char d = s.charAt(left);
                window[d-'a']--;
                left++;
            }
            if(isSame(need,window)){
                res.add(left);
            }
        }
        return res;
    }

    private boolean isSame(int[] need, int[] window){
        for(int i=0; i<26; i++){
            if(need[i] != window[i]) return false;
        }
        return true;
    }
}
```

LeetCode567. 字符串排列

> 给你两个字符串 `s1` 和 `s2` ，写一个函数来判断 `s2` 是否包含 `s1` 的 排列。如果是，返回 `true` ；否则，返回 `false` 。
>
> 换句话说， `s1` 的排列之一是 `s2` 的 **子串** 。

示例 1：

输入： s1 = "ab" s2 = "eidbaooo"
输出： true
解释： s2 包含 s1 的排列之一 ("ba").

代码块

```
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        int[] need = new int[26];
        int[] window = new int[26];
        for(char c : s1.toCharArray()) need[c-'a']++;
        int left = 0;
        int right = 0;
        while(right < s2.length()){
            char c = s2.charAt(right);
            right++;
            window[c-'a']++;
            while(right-left > s1.length()){
                char d = s2.charAt(left);
                left++;
                window[d-'a']--;
            }
            if(isSame(need,window)) return true;
        }
        return false;
    }
    private boolean isSame(int[] need, int[] window){
        for(int i=0;i<26;i++){
            if(need[i] != window[i]) return false;
        }
        return true;
    }
}
```

LeetCode76. 最小覆盖子串

> 给定两个字符串 `s` 和 `t`，长度分别是 `m` 和 `n`，返回 s 中的 **最短窗口** 子串，使得该子串包含 `t` 中的每一个字符（**包括重复字符**）。如果没有这样的子串，返回空字符串 `""`。
>
> 测试用例保证答案唯一。

示例 1：

输入：s = "ADOBECODEBANC", t = "ABC"
输出："BANC"
解释：最小覆盖子串 "BANC" 包含来自字符串 t 的 'A'、'B' 和 'C'。

思路解析：滑动窗口+哈希表+valid

1. 用right指针扩大窗口
2. 用left指针缩小窗口，直到不满足条件
3. 过程中记录最小窗口

代码块

```
class Solution {
    public String minWindow(String s, String t) {
        HashMap<Character, Integer> need = new HashMap<>();
        HashMap<Character, Integer> window = new HashMap<>();
        for(char c : t.toCharArray()){
            need.put(c,need.getOrDefault(c,0)+1);
        }
        int left = 0;
        int right = 0;
        int valid = 0;
        int start = 0;
        int minLen = Integer.MAX_VALUE;
        while(right < s.length()){
            char c = s.charAt(right);
            right++;
            if(need.containsKey(c)){
                window.put(c,window.getOrDefault(c,0)+1);
                if(need.get(c).equals(window.get(c))) valid++;
            }
            while(valid == need.size()){
                if(minLen > right-left){
                    minLen = right-left;
                    start = left;
                }
                char d = s.charAt(left);
                left++;
                if(need.containsKey(d)){
                    if(window.get(d).equals(need.get(d))) valid--;
                    window.put(d,window.get(d)-1);
```

```
30                  }
31              }
32          }
33          return minLen == Integer.MAX_VALUE ? "" :
    s.substring(start,start+minLen);
34      }
35  }
```

LeetCode209. 长度最小的子数组【华为】

> 给定一个含有 `n` 个正整数的数组和一个正整数 `target` 。
>
> 找出该数组中满足其总和大于等于 `target` 的长度最小的 子数组 `[nums(l), nums(l+1), ..., nums(r-1), nums(r)]` ，并返回其长度。如果不存在符合条件的子数组，返回 `0` 。

示例 1：

输入： target = 7, nums = [2,3,1,2,4,3]
输出： 2
解释： 子数组 [4,3] 是该条件下的长度最小的子数组。

思路解析： 滑动窗口

代码块

```
1   class Solution {
2       public int minSubArrayLen(int target, int[] nums) {
3           int left = 0;
4           int minLen = Integer.MAX_VALUE;
5           int curSum = 0;
6           for(int right = 0; right < nums.length; right++){
7               curSum += nums[right];
8               while(curSum >= target){
9                   minLen = Math.min(minLen, right-left+1);
10                  curSum -= nums[left];
11                  left++;
12              }
13          }
14          return minLen == Integer.MAX_VALUE ? 0 : minLen;
15      }
16  }
```

2. 双指针

LeetCode75. 颜色分类

> 给定一个包含红色、白色和蓝色、共 `n` 个元素的数组 `nums` ，**原地** 对它们进行排序，使得相同颜色的元素相邻，并按照红色、白色、蓝色顺序排列。
>
> 我们使用整数 `0` 、 `1` 和 `2` 分别表示红色、白色和蓝色。
>
> 必须在不使用库内置的 sort 函数的情况下解决这个问题。

示例 1：

输入：nums = [2,0,2,1,1,0]
输出：[0,0,1,1,2,2]

解法思路：三指针

代码块

```
public void sortColors(int[] nums) {
        int n = nums.length;
        int p0 = 0;
        int p2 = n-1;
        int cur = 0;
        while(cur <= p2){
            if(nums[cur] == 0){
                swap(nums,p0,cur);
                p0++;
                cur++;
            } else if(nums[cur] == 2){
                swap(nums,cur,p2);
                p2--;
            } else {
                cur++;
            }
        }
    }

private void swap(int[] nums, int i, int j){
       int temp = nums[i];
       nums[i] = nums[j];
       nums[j] = temp;
}
```

LeetCode647. 回文子串

> 给你一个字符串 `s` ，请你统计并返回这个字符串中 **回文子串** 的数目。
>
> **回文字符串** 是正着读和倒过来读一样的字符串。
>
> **子字符串** 是字符串中的由连续字符组成的一个序列。

示例 1：

输入：s = "abc"
输出：3
解释：三个回文子串: "a", "b", "c"

解析思路：中心拓展法

代码块

```
class Solution {
    int cnt = 0;
    public int countSubstrings(String s) {
        for(int i=0; i<s.length(); i++){
            expand(s,i,i);
            expand(s,i,i+1);
        }
        return cnt;
    }
    private void expand(String s, int i, int j){
        while(i>=0 && j<s.length() && s.charAt(i) == s.charAt(j)){
            i--;
            j++;
            cnt++;
        }
        return;
    }
}
```

LeetCode26. 删除有序数组中的重复项

> 给你一个 **非严格递增排列** 的数组 `nums` ，请你 **原地** 删除重复出现的元素，使每个元素 **只出现一次** ，返回删除后数组的新长度。元素的 **相对顺序** 应该保持 **一致** 。然后返回 `nums` 中唯一元素的个数。
>
> 考虑 `nums` 的唯一元素的数量为 `k` 。去重后，返回唯一元素的数量 `k` 。
>
> `nums` 的前 `k` 个元素应包含 **排序后** 的唯一数字。下标 `k - 1` 之后的剩余元素可以忽略。

示例 1：

输入：nums = [1,1,2]
输出：2, nums = [1,2,_]
解释：函数应该返回新的长度 **2** ，并且原数组 *nums* 的前两个元素被修改为 **1**, **2** 。不需要考虑数组中超出新长度后面的元素。

解法思路：快慢指针

代码块

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int slow = 0;
        for(int fast = 1; fast<nums.length; fast++){
            if(nums[slow]==nums[fast]) continue;
            else {
                slow++;
                nums[slow] = nums[fast];
            }
        }
        return slow+1;
    }
}
```

LeetCode80. 删除有序数组中的重复项II

> 给你一个有序数组 `nums` ，请你 **原地** 删除重复出现的元素，使得出现次数超过两次的元素**只出现两次** ，返回删除后数组的新长度。
>
> 不要使用额外的数组空间，你必须在 **原地 修改输入数组** 并在使用 O(1) 额外空间的条件下完成。

示例 1：

输入：nums = [1,1,1,2,2,3]
输出：5, nums = [1,1,2,2,3]
解释：函数应返回新长度 length = `5` , 并且原数组的前五个元素被修改为 `1, 1, 2, 2, 3` 。 不需要考虑数组中超出新长度后面的元素。

解法思路：快慢指针

代码块

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int n = nums.length;
```

```
4           if(n<=2) return n;
5           int slow = 2;
6           for(int fast=2; fast<n; fast++){
7               if(nums[fast]!=nums[slow-2]){
8                   nums[slow] = nums[fast];
9                   slow++;
10              }
11          }
12          return slow;
13      }
14  }
```

LeetCode287. 寻找重复数

> 给定一个包含 `n + 1` 个整数的数组 `nums` ，其数字都在 `[1, n]` 范围内（包括 `1` 和 `n`），可知至少存在一个重复的整数。
>
> 假设 `nums` 只有 **一个重复的整数** ，返回 **这个重复的数** 。
>
> 你设计的解决方案必须 **不修改** 数组 `nums` 且只用常量级 `O(1)` 的额外空间。

示例 1：

输入： nums = [1,3,4,2,2]
输出： 2

思路解析：快慢指针

```
代码块
1
```

LeetCode15. 三数之和【华为】

> 给你一个整数数组 `nums` ，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k` ，同时还满足 `nums[i] + nums[j] + nums[k] == 0` 。请你返回所有和为 `0` 且不重复的三元组。
>
> **注意：**答案中不可以包含重复的三元组。

示例 1：

输入： nums = [-1,0,1,2,-1,-4]
输出： [[-1,-1,2],[-1,0,1]]

解释：
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0 。
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0 。
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0 。
不同的三元组是 [-1,0,1] 和 [-1,-1,2] 。
注意，输出的顺序和三元组的顺序并不重要。

代码块

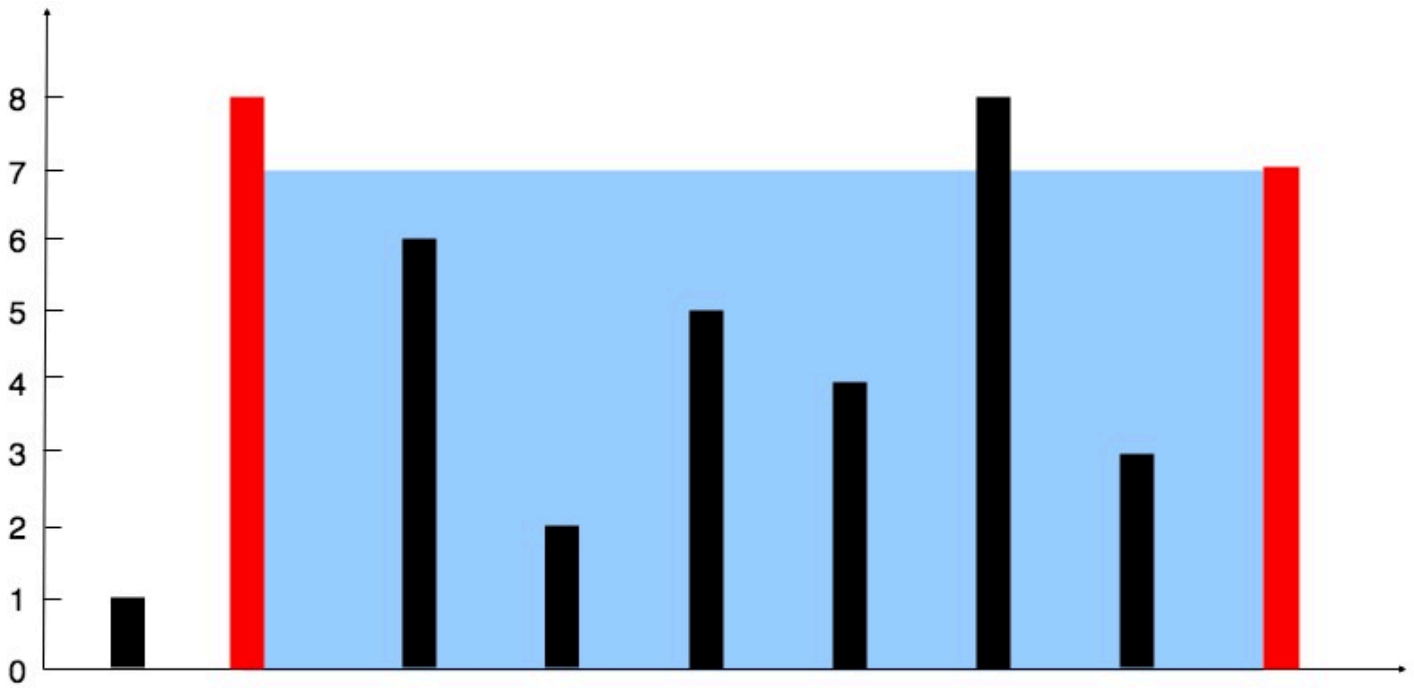
```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        int n = nums.length;
        Arrays.sort(nums);
        for(int i=0; i<n; i++){
            if(nums[i] > 0) break;
            if(i>0 && nums[i] == nums[i-1]) continue;
            int left = i+1;
            int right = n-1;
            while(left < right){
                int sum = nums[i]+nums[left]+nums[right];
                if(sum == 0){
                    res.add(Arrays.asList(nums[i],nums[left],nums[right]));
                    while(left < right && nums[left] == nums[left+1]) left++;
                    while(left < right && nums[right] == nums[right-1]) right--;
                    left++;
                    right--;
                } else if(sum < 0){
                    left++;
                } else {
                    right--;
                }
            }
        }
        return res;
    }
}
```

LeetCode11. 盛最多水的容器【华为】

> 给定一个长度为 `n` 的整数数组 `height` 。有 `n` 条垂线，第 `i` 条线的两个端点是 `(i, 0)` 和 `(i, height[i])` 。

> 找出其中的两条线，使得它们与 `x` 轴共同构成的容器可以容纳最多的水。
>
> 返回容器可以储存的最大水量。
>
> **说明：**你不能倾斜容器。

示例 1：



输入：[1,8,6,2,5,4,8,3,7]
输出：49
解释：图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下，容器能够容纳水（表示为蓝色部分）的最大值为 49。

代码块

```
class Solution {
    public int maxArea(int[] height) {
        int left = 0;
        int right = height.length-1;
        int max = 0;
        while(left < right){
            int w = right-left;
            int h = Math.min(height[left], height[right]);
            max = Math.max(max,w*h);
            if(height[left] > height[right]){
                right--;
            } else {
                left++;
            }
        }
        return max;
    }
```

```
18    }
```

LeetCode42. 接雨水【华为】

> 给定 `n` 个非负整数表示每个宽度为 `1` 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1：



输入： height = [0,1,0,2,1,0,1,3,2,1,2,1]
输出： 6
解释： 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

双指针解法：

代码块

```
1   class Solution {
2       public int trap(int[] height) {
3           int left = 0;
4           int leftMax = 0;
5           int right = height.length-1;
6           int rightMax = 0;
7           int sum = 0;
8           while(left < right){
9               if(height[left] < height[right]){
10                  if(height[left] > leftMax){
11                      leftMax = height[left];
12                  } else {
13                      sum += leftMax-height[left];
14                  }
15                  left++;
16              } else {
17                  if(height[right] > rightMax){
18                      rightMax = height[right];
19                  } else {
```

```
                    sum += rightMax-height[right];
                }
                right--;
            }
        }
        return sum;
    }
}
```

单调递减栈解法：

维护一个严格单调递减的栈（栈中存储数组索引，对应高度递减）。

雨水高度 = min(左边界高度, 右边界高度) - 凹槽底部高度

雨水宽度 = 右边界索引 - 左边界索引 - 1

当前层雨水量 = 雨水高度 × 雨水宽度

单调递减栈

```
class Solution {
    public int trap(int[] height) {
        int sum = 0;
        // 单调递减栈：存储数组索引，对应高度递减
        Deque<Integer> stack = new LinkedList<>();

        for(int i=0; i<height.length; i++){
            // 当当前高度大于栈顶高度时，形成凹槽，计算雨水
            while(!stack.isEmpty() && height[i]>height[stack.peek()]){
                int bottomIndex = stack.pop();
                // 弹出后栈为空，说明没有左边界，无法接水
                if(stack.isEmpty()) break;
                // 左边界是新的栈顶，右边界是当前元素i
                int leftIndex = stack.peek();
                int rightIndex = i;

                int waterHeight = Math.min(height[leftIndex],
height[rightIndex]) - height[bottomIndex];
                int waterWidth = rightIndex - leftIndex - 1;

                sum += waterHeight*waterWidth;
            }
            // 当前高度小于等于栈顶高度，入栈保持单调递减
            stack.push(i);
        }
        return sum;
    }
```

```
27  }
```

LeetCode189. 轮转数组

> 给定一个整数数组 `nums` ，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。

示例 1:

代码块

```
1  输入: nums = [1,2,3,4,5,6,7], k = 3
2  输出: [5,6,7,1,2,3,4]解释:
3  向右轮转 1 步: [7,1,2,3,4,5,6]
4  向右轮转 2 步: [6,7,1,2,3,4,5]
5  向右轮转 3 步: [5,6,7,1,2,3,4]
```

解法：三次反转

代码块

```
1  class Solution {
2      public void rotate(int[] nums, int k) {
3          int n = nums.length;
4          k = k % n;
5          if (k == 0) return;
6
7          reverse(nums, 0, n - 1);    // 整体反转
8          reverse(nums, 0, k - 1);    // 反转前k个
9          reverse(nums, k, n - 1);    // 反转剩下部分
10     }
11
12     // 反转 [l, r] 闭区间
13     private void reverse(int[] nums, int l, int r) {
14         while (l < r) {
15             int tmp = nums[l];
16             nums[l] = nums[r];
17             nums[r] = tmp;
18             l++;
19             r--;
20         }
21     }
22 }
```

3. 数组

LeetCode238. 除自身以外数组的乘积

> 给你一个整数数组 `nums` ，返回 数组 `answer` ，其中 `answer[i]` 等于 `nums` 中除了 `nums[i]` 之外其余各元素的乘积 。
>
> 题目数据 **保证** 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内。
>
> 请 **不要使用除法，**且在 `O(n)` 时间复杂度内完成此题。

示例 1:

输入: nums = [1,2,3,4]

输出: [24,12,8,6]

解法思路：左右乘积法

代码块

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        res[0] = 1;
        for(int i=1; i<n; i++){
            res[i] = nums[i-1]*res[i-1];
        }
        int val = 1;
        for(int i=n-1; i>=0; i--){
            res[i] = res[i]*val;
            val = val*nums[i];
        }
        return res;
    }
}
```

LeetCode581. 最短无序连续子数组

> 给你一个整数数组 `nums` ，你需要找出一个 **连续子数组** ，如果对这个子数组进行升序排序，那么整个数组都会变为升序排序。
>
> 请你找出符合题意的 **最短** 子数组，并输出它的长度。

示例 1：

输入： nums = [2,6,4,8,10,9,15]
输出： 5
解释： 你只需要对 [6, 4, 8, 10, 9] 进行升序排序，那么整个表都会变为升序排序。

解法思路：从左往右找最后一个破坏升序位置的（右边界）；从右往左找第一个破坏升序的位置（左边界）

代码块

```java
class Solution {
    public int findUnsortedSubarray(int[] nums) {
        int n = nums.length;
        int left = 0;
        int right = -1;
        int min = Integer.MAX_VALUE;
        int max = Integer.MIN_VALUE;
        for(int i=0; i<n; i++){ //找右边界最后一个比max小的位置
            if(nums[i] >= max) max = nums[i];
            else right = i;
        }
        for(int i=n-1; i>=0; i--){ //找左边界第一个比min大的位置
            if(nums[i] <= min) min = nums[i];
            else left = i;
        }
        return right-left+1;
    }
}
```

LeetCode215. 数组中的第k大元素

> 给定整数数组 `nums` 和整数 `k` ，请返回数组中第 `k` 个最大的元素。
>
> 请注意，你需要找的是数组排序后的第 `k` 个最大的元素，而不是第 `k` 个不同的元素。
>
> 你必须设计并实现时间复杂度为 `O(n)` 的算法解决此问题。

示例 1:

输入: [3,2,1,5,6,4], k = 2
输出: 5

思路解析：有限队列，大根堆

代码块

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> minHead = new PriorityQueue<>();
        for(int num : nums){
            if(minHead.size() < k){
                minHead.offer(num);
            } else {
                if(minHead.peek()<num){
                    minHead.poll();
                    minHead.offer(num);
                }
            }
        }
        return minHead.peek();
    }
}
```

LeetCode88. 合并两个有序数组

> 给你两个按 **非递减顺序** 排列的整数数组 `nums1` 和 `nums2`，另有两个整数 `m` 和 `n`，分别表示 `nums1` 和 `nums2` 中的元素数目。
>
> 请你 **合并** `nums2` 到 `nums1` 中，使合并后的数组同样按 **非递减顺序** 排列。
>
> **注意：**最终，合并后数组不应由函数返回，而是存储在数组 `nums1` 中。为了应对这种情况，`nums1` 的初始长度为 `m + n`，其中前 `m` 个元素表示应合并的元素，后 `n` 个元素为 `0`，应忽略。`nums2` 的长度为 `n`。

代码块

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int id1 = 0;
        int id2 = 0;
        int idx = 0;
        int[] r = new int[m+n];
        while(id1<m && id2<n){
            if(nums1[id1] < nums2[id2]){
                r[idx++] = nums1[id1++];
            } else {
                r[idx++] = nums2[id2++];
```

```
            }
        }
        while(id1 < m) r[idx++] = nums1[id1++];
        while(id2 < n) r[idx++] = nums2[id2++];
        for(int i=0;i<m+n;i++) nums1[i] = r[i];
        return;
    }
}
```

LeetCode912. 排序数组-快排【华为】

> 给你一个整数数组 `nums` ，请你将该数组升序排列。
>
> 你必须在 **不使用任何内置函数** 的情况下解决问题，时间复杂度为 `O(nlog(n))` ，并且空间复杂度尽可能小。

示例 1：

输入： nums = [5,2,3,1]
输出： [1,2,3,5]
解释： 数组排序后，某些数字的位置没有改变（例如，2 和 3），而其他数字的位置发生了改变（例如，1 和 5）。

代码块

```
class Solution {

    public void quick_sort(int[] nums, int l, int r){
        if(l >= r) return;
        int x = nums[l+r >> 1];
        int i = l-1;
        int j = r+1;
        while(i < j){
            do i++; while(nums[i] < x);
            do j--; while(nums[j] > x);
            if(i < j) {
                int tmp = nums[i];
                nums[i] = nums[j];
                nums[j] = tmp;
            }
        }
        quick_sort(nums,l,j);
        quick_sort(nums,j+1,r);
    }

```

```
21      public int[] sortArray(int[] nums) {
22          int n = nums.length;
23          quick_sort(nums,0,n-1);
24          return nums;
25      }
26  }
```

LeetCode912. 排序数组-归并

> 给你一个整数数组 `nums` ，请你将该数组升序排列。
>
> 你必须在 **不使用任何内置函数** 的情况下解决问题，时间复杂度为 `O(nlog(n))` ，并且空间复杂度尽可能小。

示例 1：

输入： nums = [5,2,3,1]
输出： [1,2,3,5]
解释： 数组排序后，某些数字的位置没有改变（例如，2 和 3），而其他数字的位置发生了改变（例如，1 和 5）。

代码块

```
1   class Solution {
2       int[] temp = new int[50000];
3       public void merge_sort(int[] nums, int l, int r){
4           if(l>=r) return;
5           int mid = l+r>>1;
6           merge_sort(nums,l,mid);
7           merge_sort(nums,mid+1,r);
8           int k = 0;
9           int i = l;
10          int j = mid+1;
11          while(i<=mid && j<=r){
12              if(nums[i] <= nums[j]){
13                  temp[k++] = nums[i++];
14              } else {
15                  temp[k++] = nums[j++];
16              }
17          }
18          while(i<=mid) temp[k++] = nums[i++];
19          while(j<=r) temp[k++] = nums[j++];
20          for(int w=l,m=0; w<=r; w++,m++){
21              nums[w] = temp[m];
22          }
```

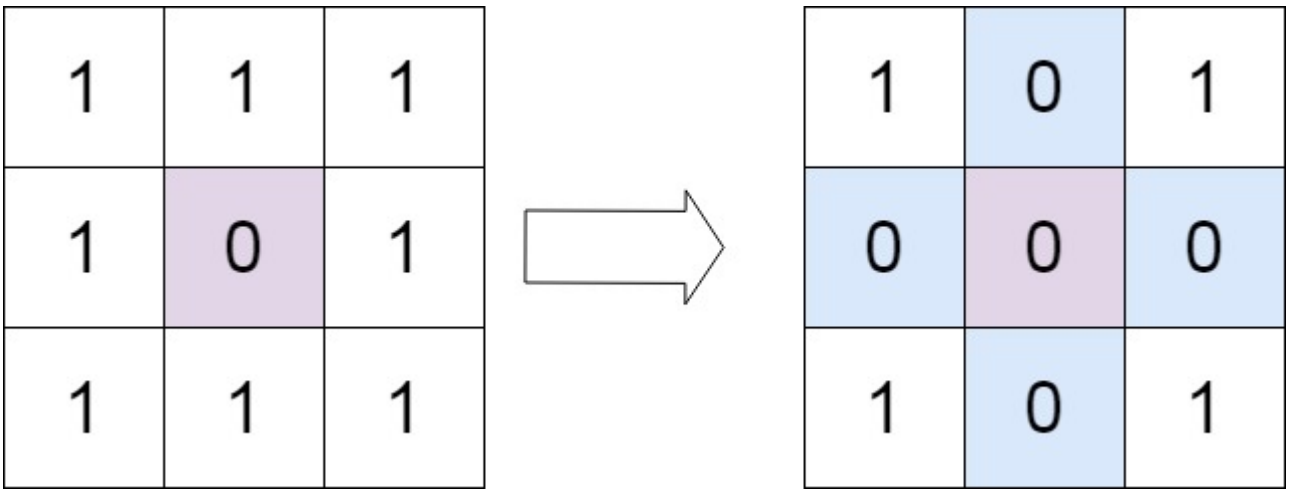
```
    }

    public int[] sortArray(int[] nums) {
        int n = nums.length;
        merge_sort(nums,0,n-1);
        return nums;
    }
}
```

LeetCode73. 矩阵置零

> 给定一个 `m x n` 的矩阵，如果一个元素为 **0** ，则将其所在行和列的所有元素都设为 **0** 。请使用 **原地** 算法。

示例 1：



输入： matrix = [[1,1,1],[1,0,1],[1,1,1]]
输出： [[1,0,1],[0,0,0],[1,0,1]]

代码块

```
class Solution {
    public void setZeroes(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        Set<Integer> row = new HashSet<>();
        Set<Integer> col = new HashSet<>();
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){
                if(matrix[i][j] == 0){
                    row.add(i);
                    col.add(j);
                }
```

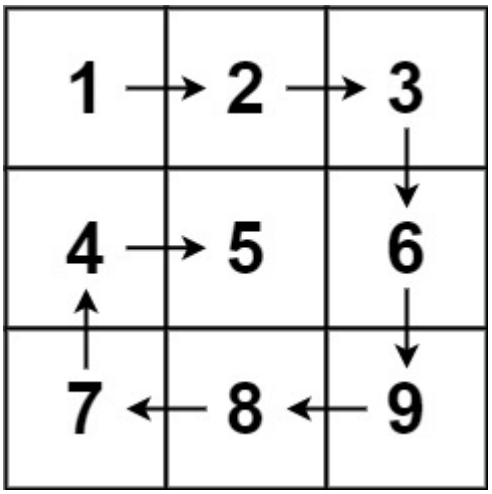
```
            }
        }

        for(int r : row){
            for(int j=0;j<n;j++){
                matrix[r][j] = 0;
            }
        }
        for(int c : col){
            for(int i=0;i<m;i++){
                matrix[i][c] = 0;
            }
        }
        return;
    }
}
```

LeetCode54. 螺旋矩阵【华为】

> 给你一个 `m` 行 `n` 列的矩阵 `matrix` ，请按照 **顺时针螺旋顺序** ，返回矩阵中的所有元素。

示例 1：



输入： matrix = [[1,2,3],[4,5,6],[7,8,9]]
输出： [1,2,3,6,9,8,7,4,5]

代码块

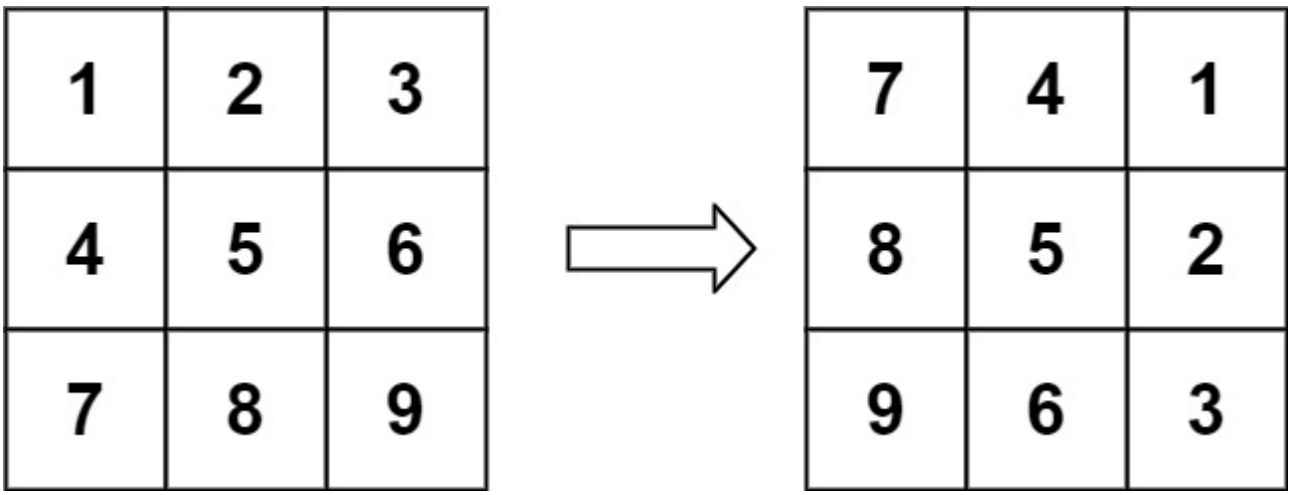
```
class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        List<Integer> res = new ArrayList<>();
```

```
6           boolean[][] flag = new boolean[m][n];
7
8           int[] dx = {0,1,0,-1};
9           int[] dy = {1,0,-1,0};
10          for(int x=0,y=0,d=0,k=1;k<=m*n;k++){
11              res.add(matrix[x][y]);
12              flag[x][y] = true;
13              int a = x + dx[d];
14              int b = y + dy[d];
15              if(a<0 || a>=m || b<0 || b>=n || flag[a][b]){
16                  d = (d+1)%4;
17                  a = x+dx[d];
18                  b = y+dy[d];
19              }
20              x = a;
21              y = b;
22          }
23          return res;
24      }
25  }
```

LeetCode48. 旋转图像

> 给定一个 *n* × *n* 的二维矩阵 `matrix` 表示一个图像。请你将图像顺时针旋转 90 度。
>
> 你必须在 **原地** 旋转图像，这意味着你需要直接修改输入的二维矩阵。**请不要** 使用另一个矩阵来旋转图像。

示例 1：



输入： matrix = [[1,2,3],[4,5,6],[7,8,9]]
输出： [[7,4,1],[8,5,2],[9,6,3]]

思路解析：先转置后每行翻转

代码块

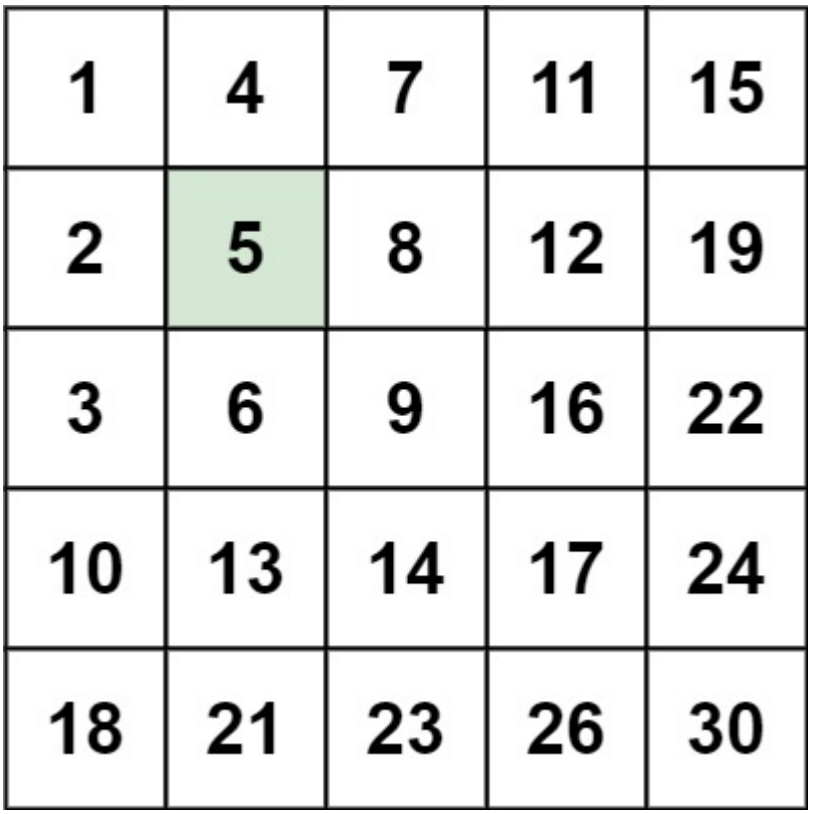
```
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        for(int i=0;i<n;i++){ //转置
            for(int j=i;j<n;j++){
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }

        for(int i=0;i<n;i++){ //反转每一行
            int left = 0;
            int right = n-1;
            while(left < right){
                int temp = matrix[i][left];
                matrix[i][left] = matrix[i][right];
                matrix[i][right] = temp;
                left++;
                right--;
            }
        }
    }
}
```

LeetCode240. 搜索二维矩阵II

> 编写一个高效的算法来搜索 `m x n` 矩阵 `matrix` 中的一个目标值 `target` 。该矩阵具有以下特性：
>
> - 每行的元素从左到右升序排列。
> - 每列的元素从上到下升序排列。

示例 1：



输入： matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5

输出： true

思路解析：Z字形搜索，从右上角开始

代码块

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length;
        int n = matrix[0].length;
        int row = 0;
        int col = n-1;
        while(row<m && col>=0){
            int cur = matrix[row][col];
            if(cur == target){
                return true;
            } else if(cur > target){
                col--;
            } else {
                row++;
            }
        }
        return false;
    }
}
```

LeetCode136. 只出现一次的数字

> 给你一个 **非空** 整数数组 `nums` ，除了某个元素只出现一次以外，其余每个元素均出现两次。找出那个只出现了一次的元素。
>
> 你必须设计并实现线性时间复杂度的算法来解决此问题，且该算法只使用常量额外空间。

示例 1：

输入：nums = [2,2,1]

输出：1

思路解析：异或运算

异或 `^` 三大性质：

1. 任何数 **异或自身 = 0** `a ^ a = 0`
2. 任何数 **异或 0 = 自身** `a ^ 0 = a`
3. 异或满足**交换律、结合律**

代码块

```
class Solution {
    public int singleNumber(int[] nums) {
        int result = 0;
        for(int num : nums){
            result ^= num;
        }
        return result;
    }
}
```

LeetCode31. 下一个排列

> 整数数组的一个 **排列** 就是将其所有成员以序列或线性顺序排列。
>
> - 例如， `arr = [1,2,3]` ，以下这些都可以视作 `arr` 的排列： `[1,2,3]` 、 `[1,3,2]` 、 `[3,1,2]` 、 `[2,3,1]` 。
>
> 整数数组的 **下一个排列** 是指其整数的下一个字典序更大的排列。更正式地，如果数组的所有排列根据其字典顺序从小到大排列在一个容器中，那么数组的 **下一个排列** 就是在这个有序容器中排在它后面的那个排列。如果不存在下一个更大的排列，那么这个数组必须重排为字典序最小的排列（即，其元素按升序排列）。

> - 例如，`arr = [1,2,3]` 的下一个排列是 `[1,3,2]` 。
> - 类似地，`arr = [2,3,1]` 的下一个排列是 `[3,1,2]` 。
> - 而 `arr = [3,2,1]` 的下一个排列是 `[1,2,3]` ，因为 `[3,2,1]` 不存在一个字典序更大的排列。
>
> 给你一个整数数组 `nums` ，找出 `nums` 的下一个排列。
>
> 必须 **原地** 修改，只允许使用额外常数空间。

示例 1：

输入：nums = [1,2,3]
输出：[1,3,2]

核心算法（固定 4 步模板，背下来就能秒写）

1. **从后往前**找第一个下标 `i` ，满足：`nums[i] < nums[i+1]`
2. 找不到说明整体降序，**直接反转整个数组**
3. 再**从后往前**找第一个下标 `j` ，满足：`nums[j] > nums[i]`
4. 交换 `nums[i]` 和 `nums[j]`
5. **反转 i 后面所有元素**（让后面变成升序，保证最小增幅）

代码块

```java
class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int i = n-2;
        // 1. 从后往前找第一个 nums[i] < nums[i+1]
        while(i >=  0 && nums[i] >= nums[i+1]){
            i--;
        }

        // 2. 不是最大排列，需要找交换的 j
        if(i>=0){
            int j = n - 1;
            // 从后往前找第一个比 nums[i] 大的
            while (nums[j] <= nums[i]) {
                j--;
            }
            // 交换 i j
            swap(nums, i, j);
        }

        // 3. 反转 i 后面的部分；i<0 就反转整个数组
        reverse(nums, i + 1, n - 1);
    }
```

```
    // 交换
    private void swap(int[] nums, int a, int b) {
        int temp = nums[a];
        nums[a] = nums[b];
        nums[b] = temp;
    }

    // 反转区间 [l, r]
    private void reverse(int[] nums, int l, int r) {
        while (l < r) {
            swap(nums, l, r);
            l++;
            r--;
        }
    }
}
```

LeetCode169. 多数元素【华为】

> 给定一个大小为 `n` 的数组 `nums` ，返回其中的多数元素。多数元素是指在数组中出现次数 **大于** `⌊ n/2 ⌋` 的元素。
>
> 你可以假设数组是非空的，并且给定的数组总是存在多数元素。

示例 1：

输入： nums = [3,2,3]
输出： 3

代码块

```
class Solution {
    public int majorityElement(int[] nums) {
        int candidate = 0;
        int count = 0;
        for(int i=0; i<nums.length; i++){
            if(count == 0){
                candidate = nums[i];
            }
            count += candidate == nums[i] ? 1:-1;
        }
        return candidate;
    }
}
```

LeetCode179. 最大数【华为】

> 给定一组非负整数 `nums` ，重新排列每个数的顺序（每个数不可拆分）使之组成一个最大的整数。
>
> **注意：**输出结果可能非常大，所以你需要返回一个字符串而不是整数。

示例 1：

输入：nums = [10,2]

输出："210"

核心思路：自定义排序

先转换成字符串，再自定义排序

```java
class Solution {
    public String largestNumber(int[] nums) {
        String[] strs = new String[nums.length];
        for(int i=0; i<nums.length; i++){
            strs[i] = String.valueOf(nums[i]);
        }
        // Arrays.sort(strs, (a,b)->(b+a).compareTo(a+b));
        Arrays.sort(strs, new Comparator<String>() {
            public int compare(String a, String b) {
                String ab = a + b;
                String ba = b + a;
                // 降序排列，返回ba.compareTo(ab)
                return ba.compareTo(ab);
            }
        });
        StringBuilder sb = new StringBuilder();
        for(String s : strs){
            sb.append(s);
        }
        if(sb.charAt(0) == '0')
            return "0";

        return sb.toString();
    }
}
```

4. 字符串

LeetCode14. 最长公共前缀

> 编写一个函数来查找字符串数组中的最长公共前缀。
>
> 如果不存在公共前缀，返回空字符串 `""`。

示例 1：

输入：strs = ["flower","flow","flight"]
输出："fl"

代码块

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        String str = strs[0];
        for(int i=0; i<str.length(); i++){
            char c = str.charAt(i);
            for(String s : strs){
                if(s.length() == i || s.charAt(i) != c)
                    return str.substring(0,i);
            }
        }
        return str;
    }
}
```

LeetCode151. 反转字符串中的单词

> 给你一个字符串 `s` ，请你反转字符串中 **单词** 的顺序。
>
> **单词** 是由非空格字符组成的字符串。 `s` 中使用至少一个空格将字符串中的 **单词** 分隔开。
>
> 返回 **单词** 顺序颠倒且 **单词** 之间用单个空格连接的结果字符串。
>
> **注意：**输入字符串 `s` 中可能会存在前导空格、尾随空格或者单词间的多个空格。返回的结果字符串中，单词间应当仅用单个空格分隔，且不包含任何额外的空格。

示例 1：

输入：s = "the sky is blue"
输出："blue is sky the"

代码块

```
class Solution {
    public String reverseWords(String s) {
        String[] words = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for(int i=words.length-1;i>=0;i--){
```

```
            sb.append(words[i]);
            if(i>0) sb.append(" ");
        }
        return sb.toString();
    }
}
```

LeetCode5. 最长回文子串

> 给你一个字符串 `s`，找到 `s` 中最长的 回文 子串。

示例 1：

输入： s = "babad"
输出： "bab"
解释： "aba" 同样是符合题意的答案。

代码块

```
class Solution {
    public String longestPalindrome(String s) {
        int start = 0;
        int end = 0;
        for(int i=0; i<s.length(); i++){
            int ll = expand(s,i,i);
            int lr = expand(s,i,i+1);
            int maxLen = Math.max(ll,lr);
            if(maxLen > end-start){
                start = i-(maxLen-1)/2;
                end = i+maxLen/2;
            }
        }
        return s.substring(start,end+1);
    }
    private int expand(String s, int left, int right){
        while(left>=0 && right<s.length() && s.charAt(left) == s.charAt(right))
{
            left--;
            right++;
        }
        return right-left-1;
    }
}
```

LeetCode451. 根据字符出现频率排序【华为】

> 给定一个字符串 `s` ，根据字符出现的 **频率** 对其进行 **降序排序** 。一个字符出现的 **频率** 是它出现在字符串中的次数。
>
> 返回 *已排序的字符串* 。如果有多个答案，返回其中任何一个

示例 1:

输入: s = "tree"
输出: "eert"
解释: 'e'出现两次，'r'和't'都只出现一次。
因此'e'必须出现在'r'和't'之前。此外，"eetr"也是一个有效的答案。

核心思路：桶排序

利用**频率范围有限**的特性（最大频率不超过字符串长度）：

- 创建桶数组，**下标 = 频率**，桶里存对应频率的字符
- 从最高频率倒序遍历桶，依次拼接字符

代码块

```
class Solution {
    public String frequencySort(String s) {
        // 1. 统计每个字符的出现频率
        Map<Character, Integer> countMap = new HashMap<>();
        for (char c : s.toCharArray()) {
            countMap.put(c, countMap.getOrDefault(c, 0) + 1);
        }

        // 2. 创建桶数组：下标是频率，桶中存储对应频率的字符
        // 最大频率不超过字符串长度，所以桶大小为 s.length()+1
        List<Character>[] bucket = new List[s.length() + 1];
        for (Map.Entry<Character, Integer> entry : countMap.entrySet()) {
            int freq = entry.getValue();
            if (bucket[freq] == null) {
                bucket[freq] = new ArrayList<>();
            }
            bucket[freq].add(entry.getKey());
        }

        // 3. 从最高频率倒序遍历桶，拼接结果
        StringBuilder sb = new StringBuilder();
        for (int i = bucket.length - 1; i >= 0; i--) {
            if (bucket[i] != null) {
                for (char c : bucket[i]) {
```

```
                    // 每个字符拼接 i 次（i是它的频率）
                    sb.append(String.valueOf(c).repeat(i));
                }
            }
        }

        return sb.toString();
    }
}
```

5. 链表

> **节点构造**

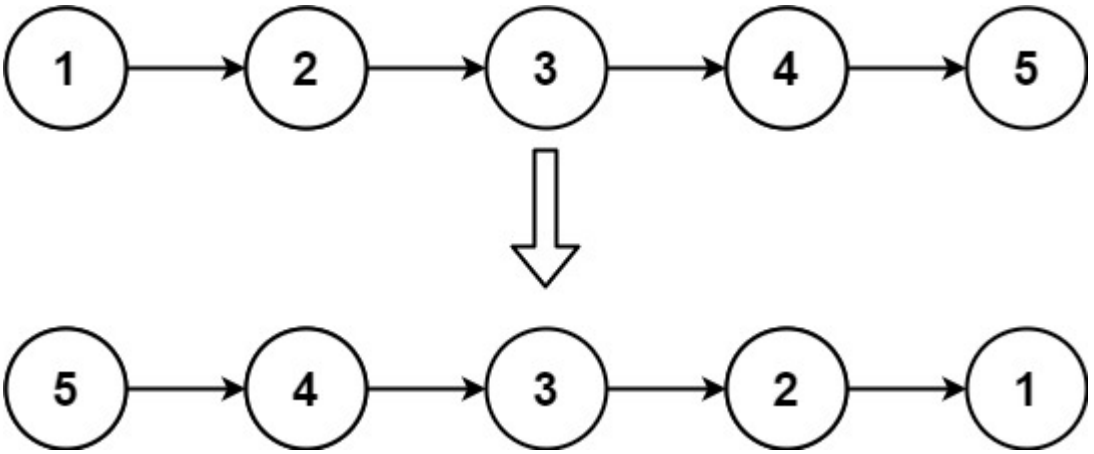
ACM模式时要带上

```
public class ListNode {
    int val;
    ListNode next;
    public ListNode(){}
    public ListNode(int val){
        this.val = val;
    }
    public ListNode(int val, ListNode next){
        this.val = val;
        this.next = next;
    }
}
```

LeetCode206. 反转链表

> 给你单链表的头节点 `head` ，请你反转链表，并返回反转后的链表。

示例 1：



输入：head = [1,2,3,4,5]
输出：[5,4,3,2,1]

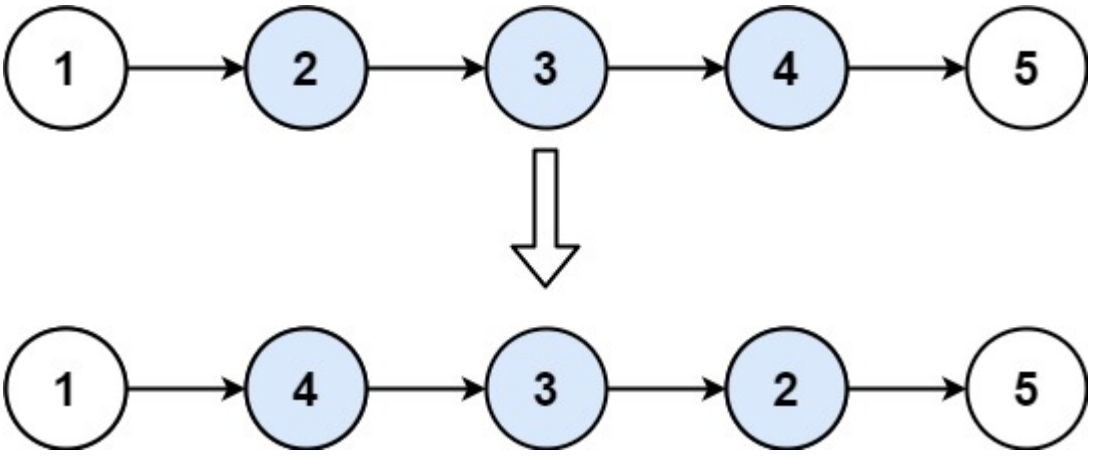
代码块

```
1   public ListNode reverseList(ListNode head) {
2           ListNode prev = null;
3           ListNode cur = head;
4           while(cur != null){
5               ListNode nextNode = cur.next;
6               cur.next = prev;
7               prev = cur;
8               cur = nextNode;
9           }
10          return prev;
11  }
```

LeetCode92. 反转链表II【华为】

> 给你单链表的头指针 `head` 和两个整数 `left` 和 `right` ，其中 `left <= right` 。请你反转从位置 `left` 到位置 `right` 的链表节点，返回 **反转后的链表** 。

示例 1：



输入：head = [1,2,3,4,5], left = 2, right = 4
输出：[1,4,3,2,5]

核心思路

1. 用**虚拟头节点**统一处理头节点反转的特殊情况
2. 先找到反转区间的**前驱节点 pre**（第 left-1 个节点）
3. 从 `pre.next` 开始，依次把后面的节点**头插**到 `pre` 的后面
4. 遍历到 right 节点时，中间区间自然反转完成

代码块

```java
class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {
        // 虚拟头节点，避免处理头节点反转的特殊情况
        ListNode dummy = new ListNode(-1);
        dummy.next = head;

        // 1. 找到反转区间的前驱节点 pre（第 left-1 个节点）
        ListNode pre = dummy;
        for (int i = 0; i < left - 1; i++) {
            pre = pre.next;
        }

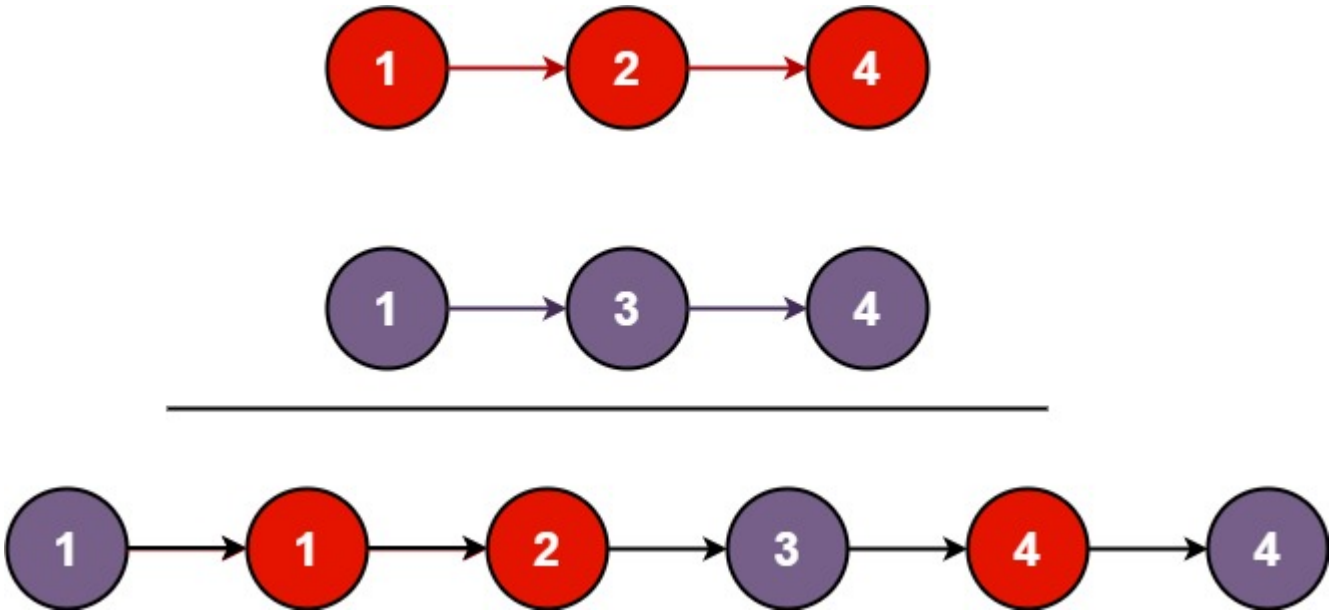
        // 2. 头插法反转区间 [left, right]
        ListNode cur = pre.next; // 反转区间的第一个节点
        // 循环 right-left 次，把后面的节点依次头插到 pre 后面
        for (int i = 0; i < right - left; i++) {
            ListNode next = cur.next; // 要移动的节点
            cur.next = next.next;     // cur 指向 next 的下一个
            next.next = pre.next;     // next 指向 pre 的下一个
            pre.next = next;          // pre 指向 next
        }

        return dummy.next;
    }
}
```

LeetCode21. 合并两个有序链表【华为】

> 将两个升序链表合并为一个新的 **升序** 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例 1：



输入： l1 = [1,2,4], l2 = [1,3,4]
输出： [1,1,2,3,4,4]

代码块

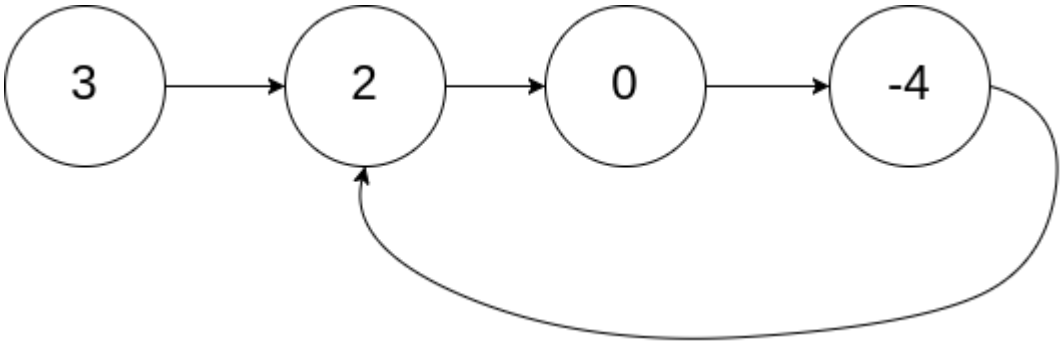
```
1   class Solution {
2       public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
3           ListNode dummy = new ListNode(-1);
4           ListNode cur = dummy;
5           while(list1 != null && list2 != null){
6               if(list1.val < list2.val){
7                   cur.next = list1;
8                   list1 = list1.next;
9               } else {
10                  cur.next = list2;
11                  list2 = list2.next;
12              }
13              cur = cur.next;
14          }
15          cur.next = list1!=null ? list1 : list2;
16          return dummy.next;
17      }
18  }
```

LeetCode141. 环形链表

> 给你一个链表的头节点 `head` ，判断链表中是否有环。
>
> 如果链表中有某个节点，可以通过连续跟踪 `next` 指针再次到达，则链表中存在环。 为了表示给定链表中的环，评测系统内部使用整数 `pos` 来表示链表尾连接到链表中的位置（索引从 0 开始）。

> **注意：** `pos` **不作为参数进行传递** 。仅仅是为了标识链表的实际情况。
>
> *如果链表中存在环* ，则返回 `true` 。 否则，返回 `false` 。

示例 1：



输入：head = [3,2,0,-4], pos = 1
输出：true
解释：链表中有一个环，其尾部连接到第二个节点。

解题思路：快慢指针

代码块

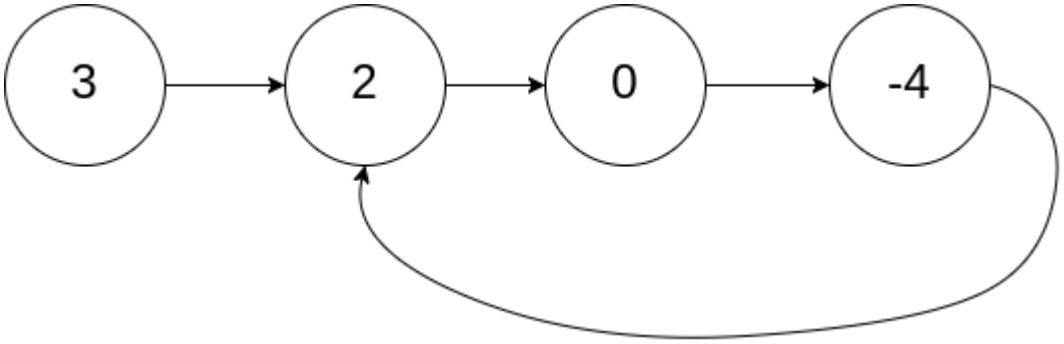
```
public class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode fast = head;
        ListNode slow = head;
        while(fast != null && fast.next != null){
            slow = slow.next;
            fast = fast.next.next;
            if(slow == fast) return true;
        }
        return false;
    }
}
```

LeetCode142. 环形链表II

> 给定一个链表的头节点 `head` ，返回链表开始入环的第一个节点。 *如果链表无环，则返回* `null` 。
>
> 如果链表中有某个节点，可以通过连续跟踪 `next` 指针再次到达，则链表中存在环。 为了表示给定链表中的环，评测系统内部使用整数 `pos` 来表示链表尾连接到链表中的位置（**索引从 0 开始**）。如果 `pos` 是 `-1` ，则在该链表中没有环。**注意：** `pos` **不作为参数进行传递**，仅仅是为了标识链表的实际情况。

> **不允许修改** 链表。

示例 1：



输入： head = [3,2,0,-4], pos = 1
输出： 返回索引为 1 的链表节点
解释： 链表中有一个环，其尾部连接到第二个节点。

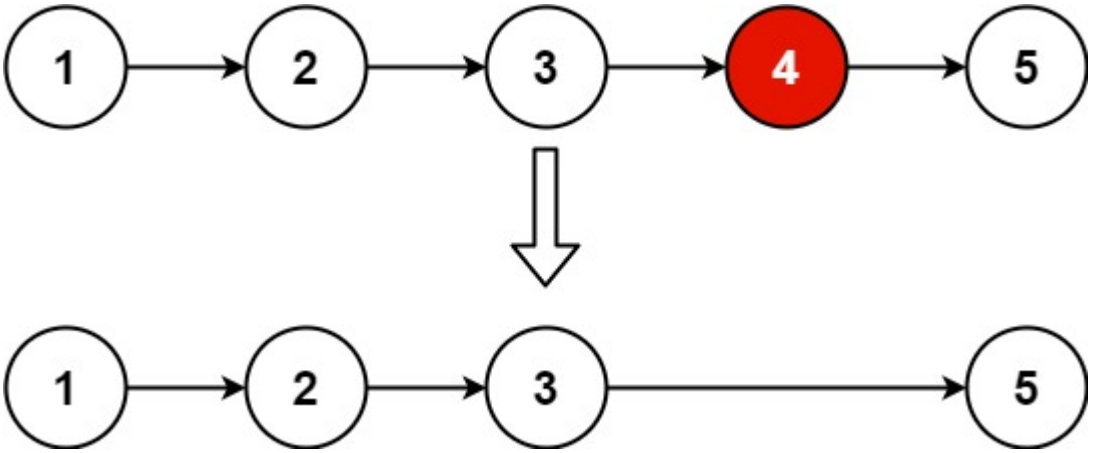
代码块

```
public class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;
        boolean hasCycle = false;
        while(fast!=null && fast.next!=null){
            slow = slow.next;
            fast = fast.next.next;
            if(fast == slow) {
                hasCycle = true;
                break;
            }
        }
        if(!hasCycle) return null;
        slow = head;
        while(slow != fast){
            slow = slow.next;
            fast = fast.next;
        }
        return slow;
    }
}
```

LeetCode19. 删除链表的倒数第n个结点【华为】

> 给你一个链表，删除链表的倒数第 `n` 个结点，并且返回链表的头结点。

示例 1：



输入： head = [1,2,3,4,5], n = 2
输出： [1,2,3,5]

代码块

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(-1);
        dummy.next = head;
        ListNode fast = dummy;
        for(int i=0;i<=n;i++) fast = fast.next;
        ListNode slow = dummy;
        while(fast != null){
            fast = fast.next;
            slow = slow.next;
        }
        slow.next = slow.next.next;
        return dummy.next;  // 注意是dummy
    }
}
```

LeetCode146. LRU 缓存⭐【华为】

请你设计并实现一个满足 LRU (最近最少使用) 缓存 约束的数据结构。

实现 `LRUCache` 类：

- `LRUCache(int capacity)` 以 **正整数** 作为容量 `capacity` 初始化 LRU 缓存
- `int get(int key)` 如果关键字 `key` 存在于缓存中，则返回关键字的值，否则返回 `-1` 。

- `void put(int key, int value)` 如果关键字 `key` 已经存在，则变更其数据值 `value` ；如果不存在，则向缓存中插入该组 `key-value` 。如果插入操作导致关键字数量超过 `capacity` ，则应该 **逐出** 最久未使用的关键字。

函数 `get` 和 `put` 必须以 `O(1)` 的平均时间复杂度运行。

示例：

输入
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
输出
[null, null, null, 1, null, -1, null, -1, 3, 4]

解释
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // 缓存是 {1=1}
lRUCache.put(2, 2); // 缓存是 {1=1, 2=2}
lRUCache.get(1); // 返回 1
lRUCache.put(3, 3); // 该操作会使得关键字 2 作废，缓存是 {1=1, 3=3}
lRUCache.get(2); // 返回 -1 (未找到)
lRUCache.put(4, 4); // 该操作会使得关键字 1 作废，缓存是 {4=4, 3=3}
lRUCache.get(1); // 返回 -1 (未找到)
lRUCache.get(3); // 返回 3
lRUCache.get(4); // 返回 4

思路解析：双向链表+哈希表

代码块

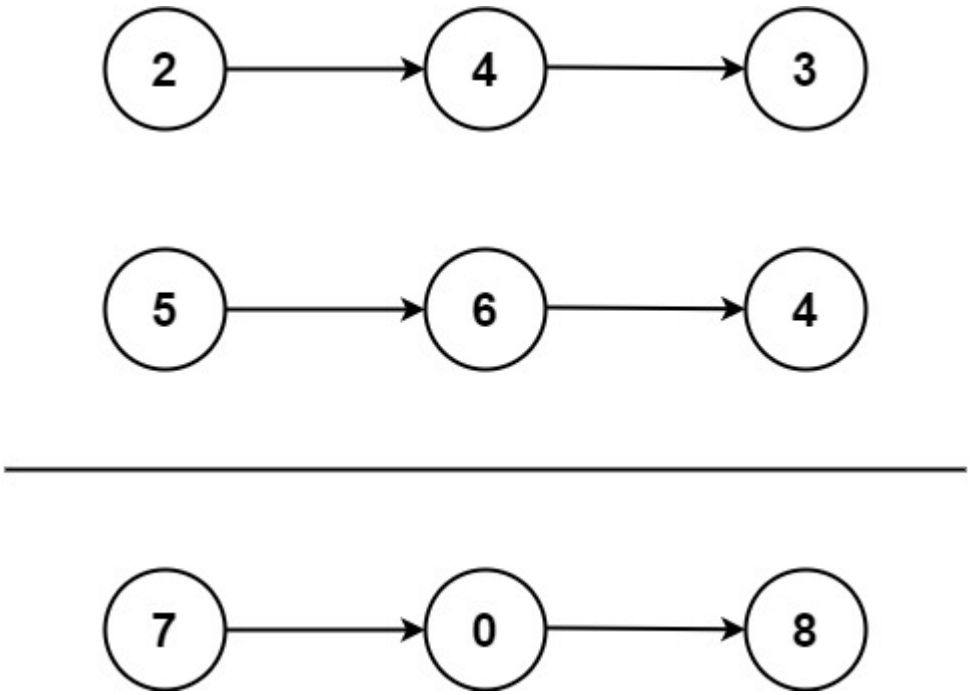
```
class DLinkedNode {
    int key;
    int val;
    DLinkedNode next;
    DLinkedNode prev;
    public DLinkedNode(){}
    public DLinkedNode(int key, int val){
        this.key = key;
        this.val = val;
    }
}
class LRUCache {
    DLinkedNode head;
    DLinkedNode tail;
    int capacity;
    int size;
```

```
17      HashMap<Integer, DLinkedNode> cache;
18
19      public LRUCache(int capacity) {
20          head = new DLinkedNode();
21          tail = new DLinkedNode();
22          head.next = tail;
23          tail.prev = head;
24          this.capacity = capacity;
25          size = 0;
26          cache = new HashMap<>();
27      }
28
29      public int get(int key) {
30          DLinkedNode node = cache.get(key);
31          if(node == null){
32              return -1;
33          } else {
34              moveToHead(node);
35              return node.val;
36          }
37      }
38
39      public void put(int key, int value) {
40          DLinkedNode node = cache.get(key);
41          if(node == null){
42              DLinkedNode newnode = new DLinkedNode(key,value);
43              addToHead(newnode);
44              cache.put(key,newnode);
45              size++;
46              if(size > capacity){
47                  DLinkedNode deNode = removeTailNode();
48                  cache.remove(deNode.key);
49                  size--;
50              }
51          } else {
52              node.val = value;
53              moveToHead(node);
54          }
55      }
56      // 用于新节点创建和被moveToHead()调用
57      private void addToHead(DLinkedNode node){
58          node.next = head.next;
59          node.prev = head;
60          head.next.prev = node;
61          head.next = node;
62      }
63      private void removeNode(DLinkedNode node){
```

```
64            node.prev.next = node.next;
65            node.next.prev = node.prev;
66        }
67        public void moveToHead(DLinkedNode node){
68            removeNode(node);
69            addToHead(node);
70        }
71        public DLinkedNode removeTailNode(){
72            DLinkedNode res = tail.prev;
73            removeNode(res);
74            return res;
75        }
76    }
```

LeetCode2. 两数相加【华为】

> 给你两个 **非空** 的链表，表示两个非负的整数。它们每位数字都是按照 **逆序** 的方式存储的，并且每个节点只能存储 **一位** 数字。
>
> 请你将两个数相加，并以相同形式返回一个表示和的链表。
>
> 你可以假设除了数字 0 之外，这两个数都不会以 0 开头。



示例 1：
输入： l1 = [2,4,3], l2 = [5,6,4]
输出： [7,0,8]
解释： 342 + 465 = 807.

```java
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(-1);
        ListNode cur = dummy;
        int carry = 0;
        while(l1 != null || l2 != null || carry != 0){
            int val1 = l1 == null ? 0 : l1.val;
            int val2 = l2 == null ? 0 : l2.val;

            int curSum = val1 + val2 + carry;
            int curVal = curSum%10;
            carry = curSum/10;

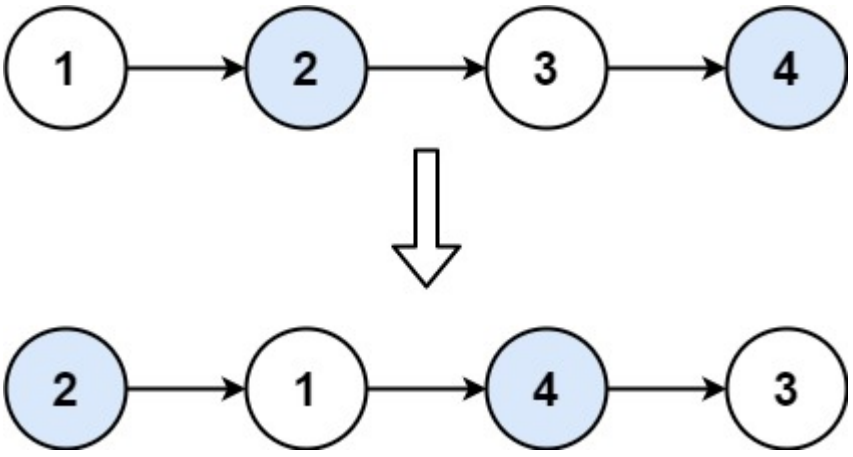
            cur.next = new ListNode(curVal);
            cur = cur.next;

            l1 = l1 == null ? null : l1.next;
            l2 = l2 == null ? null : l2.next;
        }
        return dummy.next;
    }
}
```

LeetCode24. 两两交换链表中的节点

> 给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在不修改节点内部的值的情况下完成本题（即，只能进行节点交换）。

示例 1：



输入： head = [1,2,3,4]
输出： [2,1,4,3]

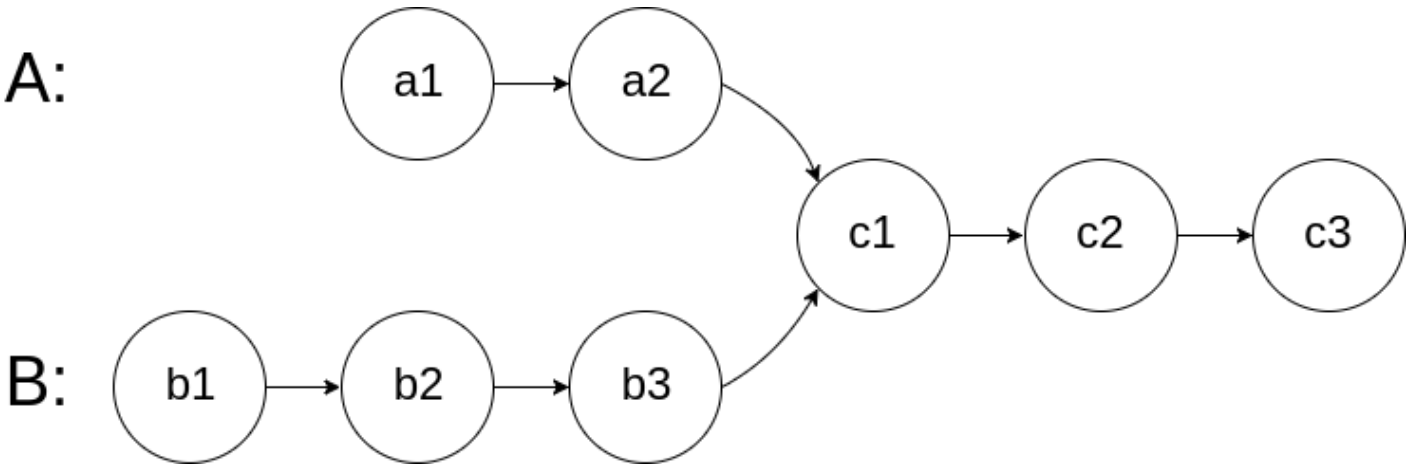
代码块

```
class Solution {
    public ListNode swapPairs(ListNode head) {
        ListNode dummy = new ListNode(-1);
        dummy.next = head;
        ListNode prev = dummy;
        while(prev.next!=null && prev.next.next!=null){
            ListNode node1 = prev.next;
            ListNode node2 = prev.next.next;

            prev.next = node2;
            node1.next = node2.next;
            node2.next = node1;

            prev = node1;
        }
        return dummy.next;
    }
}
```

LeetCode160. 相交链表

> 给你两个单链表的头节点 `headA` 和 `headB` ，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 `null` 。
>
> 图示两个链表在节点 `c1` 开始相交：
>
> 
>
>
> 题目数据 **保证** 整个链式结构中不存在环。
>
> **注意**，函数返回结果后，链表必须 **保持其原始结构** 。

代码块

```java
public class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode pA = headA;
        ListNode pB = headB;
        while(pA != pB){
            pA = pA == null ? headB : pA.next;
            pB = pB == null ? headA : pB.next;
        }
        return pA;
    }
}
```

6. 动态规划

LeetCode53. 最大子数组和【华为】

> 给你一个整数数组 `nums` ，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。
>
> 子数组是数组中的一个连续部分。

示例 1：

输入： nums = [-2,1,-3,4,-1,2,1,-5,4]
输出： 6
解释： 连续子数组 [4,-1,2,1] 的和最大，为 6 。

代码块

```java
class Solution {
    public int maxSubArray(int[] nums)
        int n = nums.length;
        if(n == 1) return nums[0];
        int[] dp = new int[n];
        dp[0] = nums[0];
        int max = dp[0];
        for(int i=1; i<n; i++){
            dp[i] = Math.max(dp[i-1]+nums[i],nums[i]);
            max = Math.max(max, dp[i]);
        }
        return max;
    }
}
```

LeetCode121. 买卖股票的最佳时机（一次买卖）

> 给定一个数组 `prices` ，它的第 `i` 个元素 `prices[i]` 表示一支给定股票第 `i` 天的价格。
>
> 你只能选择 **某一天** 买入这只股票，并选择在 **未来的某一个不同的日子** 卖出该股票。设计一个算法来计算你所能获取的最大利润。
>
> 返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 `0` 。

示例 1：

输入：[7,1,5,3,6,4]
输出：5
解释：在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = 6-1 = 5 。
注意利润不能是 7-1 = 6, 因为卖出价格需要大于买入价格；同时，你不能在买入前卖出股票。

代码块

```
class Solution {
    public int maxProfit(int[] prices) {
        int max = 0;
        int minPrice = prices[0];
        for(int i=1; i<prices.length; i++){
            minPrice = Math.min(minPrice, prices[i]);
            max = Math.max(max, prices[i]-minPrice);
        }
        return max;
    }
}
```

LeetCode309. 买卖股票的最佳时机含冷冻期

> 给定一个整数数组 `prices` ，其中第 `prices[i]` 表示第 `i` 天的股票价格 。
>
> 设计一个算法计算出最大利润。在满足以下约束条件下，你可以尽可能地完成更多的交易（多次买卖一支股票）：
>
> - 卖出股票后，你无法在第二天买入股票 (即冷冻期为 1 天)。
>
> **注意：**你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1:

输入: prices = [1,2,3,0,2]

输出: 3
解释: 对应的交易状态为: [买入, 卖出, 冷冻期, 买入, 卖出]

代码块

```
class Solution {
    public int maxProfit(int[] prices) {
        int n = prices.length;
        int[][] dp = new int[n][3];
        dp[0][0] = -prices[0]; //持有
        dp[0][1] = 0;  // 卖出
        dp[0][2] = 0;  // 不持有，不冷冻
        for(int i=1; i<n; i++){
            dp[i][0] = Math.max(dp[i-1][0],dp[i-1][2]-prices[i]);
            dp[i][1] = dp[i-1][0]+prices[i];
            dp[i][2] = Math.max(dp[i-1][2],dp[i-1][1]);
        }
        return Math.max(dp[n-1][1],dp[n-1][2]);
    }
}
```

LeetCode198. 打家劫舍

> 你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，**如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警**。
>
> 给定一个代表每个房屋存放金额的非负整数数组，计算你 **不触动警报装置的情况下** ，一夜之内能够偷窃到的最高金额。

示例 1:

输入: [1,2,3,1]
输出: 4
解释: 偷窃 1 号房屋 (金额 = 1) ，然后偷窃 3 号房屋 (金额 = 3)。偷窃到的最高金额 = 1 + 3 = 4 。

代码块

```
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if(n == 0) return 0;
        if(n < 2) return nums[0];
        int[] dp = new int[n];
        dp[0] = nums[0];
```

```
        dp[1] = Math.max(nums[0],nums[1]);
        for(int i=2; i<n; i++){
            dp[i] = Math.max(dp[i-1],dp[i-2]+nums[i]);
        }
        return dp[n-1];
    }
}
```

LeetCode213. 打家劫舍II

> 你是一个专业的小偷，计划偷窃沿街的房屋，每间房内都藏有一定的现金。这个地方所有的房屋都**围成一圈**，这意味着第一个房屋和最后一个房屋是紧挨着的。同时，相邻的房屋装有相互连通的防盗系统，**如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警**。
>
> 给定一个代表每个房屋存放金额的非负整数数组，计算你 **在不触动警报装置的情况下**，今晚能够偷窃到的最高金额。

示例 1：

输入：nums = [2,3,2]
输出：3
解释：你不能先偷窃 1 号房屋（金额 = 2），然后偷窃 3 号房屋（金额 = 2）, 因为他们是相邻的。

代码块

```
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if(n == 1) return nums[0];
        return Math.max(f(nums,0,n-2), f(nums,1,n-1));
    }
    private int f(int[] nums, int left, int right){
        int length = right-left+1;
        if(length == 1) return nums[left];
        if(length == 2) return Math.max(nums[left],nums[right]);
        int[] dp = new int[length];
        dp[0] = nums[left];
        dp[1] = Math.max(nums[left],nums[left+1]);
        for(int i=2;i<length;i++){
            dp[i] = Math.max(dp[i-1], dp[i-2]+nums[left+i]);
        }
        return dp[length-1];
    }
}
```

LeetCode300. 最长递增子序列【华为】

> 给你一个整数数组 `nums` ，找到其中最长严格递增子序列的长度。
>
> **子序列** 是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序。例如， `[3,6,2,7]` 是数组 `[0,3,1,6,2,2,7]` 的子序列。

示例 1：

输入：nums = [10,9,2,5,3,7,101,18]
输出：4
解释：最长递增子序列是 [2,3,7,101]，因此长度为 4 。

代码块

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int maxLen = 1;
        int n = nums.length;
        int[] dp = new int[n];
        for(int i=0;i<n;i++) dp[i] = 1;
        for(int i=1;i<n;i++){
            for(int j=0;j<i;j++){
                if(nums[i] > nums[j]){
                    dp[i] = Math.max(dp[i],dp[j]+1);
                }
            }
            maxLen = Math.max(maxLen,dp[i]);
        }
        return maxLen;
    }
}
```

LeetCode1143. 最长公共子序列

> 给定两个字符串 `text1` 和 `text2` ，返回这两个字符串的最长 **公共子序列** 的长度。如果不存在 **公共子序列** ，返回 `0` 。
>
> 一个字符串的 **子序列** 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。
>
> - 例如， `"ace"` 是 `"abcde"` 的子序列，但 `"aec"` 不是 `"abcde"` 的子序列。
>
> 两个字符串的 **公共子序列** 是这两个字符串所共同拥有的子序列。

示例 1：

输入：text1 = "abcde", text2 = "ace"
输出：3
解释：最长公共子序列是 "ace" ，它的长度为 3 。

二维动态规划

```java
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length();
        int n = text2.length();
        int[][] dp = new int[m+1][n+1];
        for(int i=1;i<=m;i++){
            for(int j=1;j<=n;j++){
                if(text1.charAt(i-1) == text2.charAt(j-1)){
                    dp[i][j] = Math.max(dp[i][j], dp[i-1][j-1]+1);
                } else {
                    dp[i][j] = Math.max(dp[i-1][j],dp[i][j-1]);
                }
            }
        }
        return dp[m][n];
    }
}
```

LeetCode279. 完全平方数

> 给你一个整数 `n` ，返回 *和为 `n` 的完全平方数的最少数量* 。
>
> **完全平方数** 是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如， `1` 、 `4` 、 `9` 和 `16` 都是完全平方数，而 `3` 和 `11` 不是。

示例 1：

代码块

```
输入：n = 12输出：3
解释：12 = 4 + 4 + 4
```

代码块

```java
class Solution {
    public int numSquares(int n) {
        int[] dp = new int[n+1];
```

```
        for(int i=0;i<=n;i++){
            dp[i] = i;
        }
        for(int i=1;i<=n;i++){
            for(int j=1; j*j<=i; j++){
                dp[i] = Math.min(dp[i],dp[i-j*j]+1);
            }
        }
        return dp[n];
    }
}
```

LeetCode312. 戳气球

> 有 `n` 个气球，编号为 `0` 到 `n - 1`，每个气球上都标有一个数字，这些数字存在数组 `nums` 中。
>
> 现在要求你戳破所有的气球。戳破第 `i` 个气球，你可以获得 `nums[i - 1] * nums[i] * nums[i + 1]` 枚硬币。 这里的 `i - 1` 和 `i + 1` 代表和 `i` 相邻的两个气球的序号。如果 `i - 1` 或 `i + 1` 超出了数组的边界，那么就当它是一个数字为 `1` 的气球。
>
> 求所能获得硬币的最大数量。

示例 1：

输入： nums = [3,1,5,8]
输出： 167
解释：
nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []
coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167

解法思路：区间DP

代码块

```
1
```

LeetCode322. 零钱兑换【华为】

> 给你一个整数数组 `coins` ，表示不同面额的硬币；以及一个整数 `amount` ，表示总金额。

> 计算并返回可以凑成总金额所需的 **最少的硬币个数** 。如果没有任何一种硬币组合能组成总金额，返回 `-1` 。
>
> 你可以认为每种硬币的数量是无限的。

示例 1：

代码块

```
输入：coins = [1, 2, 5], amount = 11输出：3
解释：11 = 5 + 5 + 1
```

核心思路：完全背包 动态规划

定义：`dp[i]` = 凑出金额 `i` 需要的**最少硬币数**

1. 初始化
   - `dp` 数组初始填一个大数（代表暂时凑不出）
   - `dp[0] = 0`：金额 0 不需要硬币
2. 状态转移对每个金额 `i`，遍历每种硬币：若硬币面额 ≤ 当前金额：`dp[i] = Math.min(dp[i], dp[i - coin] + 1)`

代码块

```java
class Solution {
    public int coinChange(int[] coins, int amount) {
        // 初始化dp，设为一个不可能的大数
        int[] dp = new int[amount + 1];
        int INF = amount + 1;
        for (int i = 1; i <= amount; i++) {
            dp[i] = INF;
        }
        dp[0] = 0;

        // 遍历每个金额
        for (int i = 1; i <= amount; i++) {
            // 遍历每种硬币
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }

        // 还是INF说明凑不出
        return dp[amount] == INF ? -1 : dp[amount];
    }
```

```
24  }
```

LeetCode139. 单词拆分

> 给你一个字符串 `s` 和一个字符串列表 `wordDict` 作为字典。如果可以利用字典中出现的一个或多个单词拼接出 `s` 则返回 `true`。
>
> **注意：**不要求字典中出现的单词全部都使用，并且字典中的单词可以重复使用。

示例 1：

输入: s = "leetcode", wordDict = ["leet", "code"]
输出: true
解释: 返回 true 因为 "leetcode" 可以由 "leet" 和 "code" 拼接成。

状态定义

设 `dp[i]` 表示：**字符串 s 的前 i 个字符能否被字典单词拆分**

初始化

- `dp[0] = true`：空字符串可以被拆分（基准条件）
- 其余 `dp[i]` 初始为 `false`

状态转移

对每个位置 `i`（前 i 个字符），枚举分割点 `j`：如果满足：

1. 前 `j` 个字符可以拆分：`dp[j] == true`
2. `j ~ i` 这一段子串在字典中

则 `dp[i] = true`

代码块

```
1   class Solution {
2       public boolean wordBreak(String s, List<String> wordDict) {
3           Set<String> set = new HashSet<>(wordDict);
4           int n = s.length();
5           boolean[] dp = new boolean[n + 1];
6           dp[0] = true;  // 空串合法
7
8           for (int i = 1; i <= n; i++) {
9               // 枚举分割点 j
10              for (int j = 0; j < i; j++) {
11                  if (dp[j] && set.contains(s.substring(j, i))) {
12                      dp[i] = true;
13                      break;
```

```
                }
            }
        }
        return dp[n];
    }
}
```

LeetCode62. 不同路径

> 一个机器人位于一个 `m x n` 网格的左上角 （起始点在下图中标记为 “Start” ）。
>
> 机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为 “Finish” ）。
>
> 问总共有多少条不同的路径?

示例 1：



输入： m = 3, n = 7
输出： 28

代码块

```
class Solution {
    public int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];
        // 初始化
        for(int i=0;i<m;i++) dp[i][0] = 1;
        for(int j=0;j<n;j++) dp[0][j] = 1;
        // 递推
        for(int i=1;i<m;i++){
            for(int j=1;j<n;j++){
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
            }
        }
        return dp[m-1][n-1];
    }
```

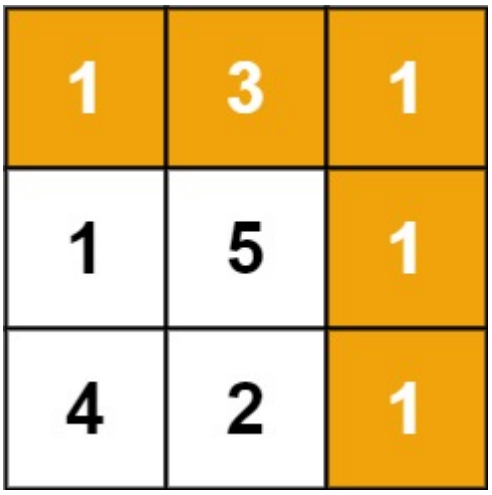
```
}

----------一维优化---------
class Solution {
    public int uniquePaths(int m, int n) {
        int[] dp = new int[n];
        // 初始全1 第一行
        for (int j = 0; j < n; j++) {
            dp[j] = 1;
        }
        // 逐行更新
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[j] = dp[j] + dp[j - 1];
            }
        }
        return dp[n - 1];
    }
}
```

LeetCode64. 最小路径和【华为】

> 给定一个包含非负整数的 `m x n` 网格 `grid` ，请找出一条从左上角到右下角的路径，使得路径上的数字总和为最小。
>
> **说明：**每次只能向下或者向右移动一步。

示例 1：



输入：grid = [[1,3,1],[1,5,1],[4,2,1]]
输出：7
解释：因为路径 1→3→1→1→1 的总和最小。

```java
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int[][] dp = new int[m][n];
        dp[0][0] = grid[0][0];
        for(int i=1; i<m; i++) dp[i][0] = grid[i][0] + dp[i-1][0];
        for(int j=1; j<n; j++) dp[0][j] = grid[0][j] + dp[0][j-1];

        for(int i=1; i<m; i++){
            for(int j=1; j<n; j++){
                dp[i][j] = grid[i][j] + Math.min(dp[i-1][j],dp[i][j-1]);
            }
        }
        return dp[m-1][n-1];
    }
}
```

LeetCode72. 编辑距离【华为】

> 给你两个单词 `word1` 和 `word2`， *请返回将 `word1` 转换成 `word2` 所使用的最少操作数* 。
>
> 你可以对一个单词进行如下三种操作：
>
> - 插入一个字符
> - 删除一个字符
> - 替换一个字符

示例 1：

输入： word1 = "horse", word2 = "ros"
输出： 3
解释：
horse -> rorse (将 'h' 替换为 'r')
rorse -> rose (删除 'r')
rose -> ros (删除 'e')

思路解析：

状态定义： 定义 `dp[i][j]` 表示：将 `word1` 的前 `i` 个字符转换成 `word2` 的前 `j` 个字符所需的最少操作次数。

初始化：

- `dp[0][j] = j`：将空字符串转换成 `word2` 的前 `j` 个字符，需要插入 `j` 个字符

- `dp[i][0] = i`：将 `word1` 的前 `i` 个字符转换成空字符串，需要删除 `i` 个字符

状态转移方程：

- 当前字符相等

如果 `word1[i-1] == word2[j-1]`（注意索引对应关系），说明这两个字符不需要任何操作，直接继承前一个子问题的解：

- 当前字符不相等

如果 `word1[i-1] != word2[j-1]`，我们可以选择三种操作中的任意一种，取操作次数最少的那个：

1. 替换操作：将 `word1[i-1]` 替换成 `word2[j-1]`，操作次数为 `dp[i-1][j-1] + 1`
2. 删除操作：删除 `word1[i-1]`，操作次数为 `dp[i-1][j] + 1`
3. 插入操作：在 `word1` 后面插入一个 `word2[j-1]`，操作次数为 `dp[i][j-1] + 1`

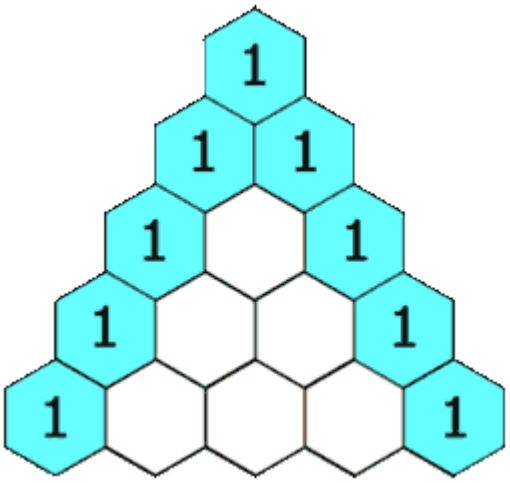
代码块

```
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length();
        int n = word2.length();
        // 创建dp数组，大小为(m+1)x(n+1)
        int[][] dp = new int[m + 1][n + 1];
        // 初始化第一列：word2为空，需要删除word1的所有字符
        for (int i = 0; i <= m; i++) {
            dp[i][0] = i;
        }
        // 初始化第一行：word1为空，需要插入word2的所有字符
        for (int j = 0; j <= n; j++) {
            dp[0][j] = j;
        }
        // 填充dp数组
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    // 当前字符相等，不需要操作
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    // 当前字符不相等，取三种操作的最小值+1
                    int replace = dp[i - 1][j - 1]; // 替换
                    int delete = dp[i - 1][j];      // 删除
                    int insert = dp[i][j - 1];      // 插入
                    dp[i][j] = Math.min(Math.min(replace, delete), insert) + 1;
                }
            }
        }
```

```
30            // 返回最终结果
31            return dp[m][n];
32        }
33    }
```

LeetCode118. 杨辉三角

> 给定一个非负整数 `numRows`，生成「杨辉三角」的前 `numRows` 行。
>
> 在**「杨辉三角」**中，每个数是它左上方和右上方的数的和。



示例 1:

输入: numRows = 5
输出: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]

代码块

```
1   class Solution {
2       public List<List<Integer>> generate(int numRows) {
3           List<List<Integer>> res = new ArrayList<>();
4           if(numRows == 0) return res;
5           int[][] dp = new int[numRows+1][numRows+1];
6           dp[1][1] = 1;
7           res.add(new ArrayList<>(List.of(1)));
8           for(int i=2; i<=numRows; i++){
9               List<Integer> cur = new ArrayList<>();
10              for(int j=1;j<=i;j++){
11                  dp[i][j] = dp[i-1][j-1]+dp[i-1][j];
12                  cur.add(dp[i][j]);
13              }
14              res.add(cur);
15          }
16          return res;
```

```
    }
}
```

7. 回溯

LeetCode46. 全排列

> 给定一个不含重复数字的数组 `nums` ，返回其 *所有可能的全排列*。你可以 **按任意顺序** 返回答案。

示例 1：

输入：nums = [1,2,3]
输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

代码块

```
class Solution {
    List<List<Integer>> res = new ArrayList<>();
    List<Integer> path = new ArrayList<>();
    boolean[] used;
    public List<List<Integer>> permute(int[] nums) {
        int n = nums.length;
        used = new boolean[n];
        backtrack(nums);
        return res;
    }
    private void backtrack(int[] nums){
        if(path.size() == nums.length){
            res.add(new ArrayList<>(path));
            return;
        }
        for(int i=0;i<nums.length;i++){
            if(used[i]) continue;
            used[i] = true;
            path.add(nums[i]);
            backtrack(nums);
            path.remove(path.size()-1);
            used[i] = false;
        }
    }
}
```

LeetCode39. 组合总数（可重复选）

> 给你一个 **无重复元素** 的整数数组 `candidates` 和一个目标整数 `target` ，找出 `candidates` 中可以使数字和为目标数 `target` 的 所有 **不同组合** ，并以列表形式返回。你可以按 **任意顺序** 返回这些组合。
>
> `candidates` 中的 **同一个** 数字可以 **无限制重复被选取** 。如果至少一个数字的被选数量不同，则两种组合是不同的。
>
> 对于给定的输入，保证和为 `target` 的不同组合数少于 `150` 个。

示例 1：

输入： candidates = [2,3,6,7], target = 7
输出： [[2,2,3],[7]]
解释：
2 和 3 可以形成一组候选，2 + 2 + 3 = 7 。注意 2 可以使用多次。
7 也是一个候选， 7 = 7 。
仅有这两种组合。

代码块

```java
class Solution {
    List<List<Integer>> res = new ArrayList<>();
    List<Integer> path = new ArrayList<>();
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        backtrack(candidates,target,0);
        return res;
    }
    private void backtrack(int[] candidates, int target, int start){
        if(target == 0){
            res.add(new ArrayList<>(path));
            return;
        }
        if(target < 0) return;  // 剪枝
        for(int i=start;i<candidates.length;i++){
            int num = candidates[i];
            path.add(num);
            backtrack(candidates,target-num,i);
            path.remove(path.size()-1);
        }
    }
}
```

LeetCode22. 括号生成

> 数字 `n` 代表生成括号的对数，请你设计一个函数，用于能够生成所有可能的并且 **有效的** 括号组合。

示例 1：

输入： n = 3
输出： ["((()))","(()())","(())()","()(())","()()()"]

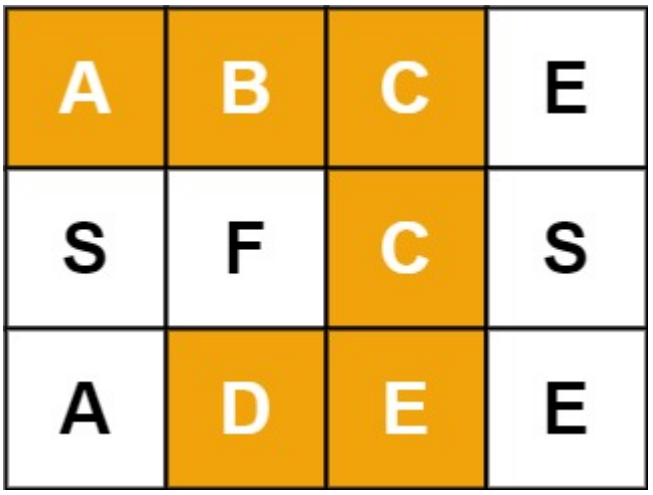
left:左括号数量；right:右括号数量

```java
class Solution {
    List<String> res = new ArrayList<>();
    public List<String> generateParenthesis(int n) {
        backtrack("",0,0,n);
        return res;
    }
    private void backtrack(String str, int left, int right, int n){
        if(str.length() == 2*n){
            res.add(str);
            return;
        }
        if(left < n) backtrack(str+"(",left+1,right,n);
        if(right < left) backtrack(str+")",left,right+1,n);
    }
}
```

LeetCode79. 单词搜索

> 给定一个 `m x n` 二维字符网格 `board` 和一个字符串单词 `word` 。如果 `word` 存在于网格中，返回 `true` ；否则，返回 `false` 。
>
> 单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中“相邻”单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使用。

示例 1：



输入： board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word = "ABCCED"
输出： true

代码块

```
class Solution {
    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
    public boolean exist(char[][] board, String word) {
        int m = board.length;
        int n = board[0].length;
        boolean[][] used = new boolean[m][n];
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){
                if(dfs(board,word,i,j,0,used)){
                    return true;
                }
            }
        }
        return false;
    }
    private boolean dfs(char[][] board, String word, int i, int j, int valid, boolean[][] used){
        if(valid == word.length()){
            return true;
        }
        if(i<0 || i>=board.length || j<0 || j>=board[0].length
        || word.charAt(valid) != board[i][j] || used[i][j]) return false;
        used[i][j] = true;
        for(int[] dir : dirs){
            int x = i + dir[0];
            int y = j + dir[1];
            if(dfs(board,word,x,y,valid+1,used)) return true;
        }
        used[i][j] = false;
        return false;
    }
```

```
31  }
```

LeetCode494. 目标和

> 给你一个非负整数数组 `nums` 和一个整数 `target` 。
>
> 向数组中的每个整数前添加 `'+'` 或 `'-'` ，然后串联起所有整数，可以构造一个 **表达式** ：
>
> - 例如， `nums = [2, 1]` ，可以在 `2` 之前添加 `'+'` ，在 `1` 之前添加 `'-'` ，然后串联起来得到表达式 `"+2-1"` 。
>
> 返回可以通过上述方法构造的、运算结果等于 `target` 的不同 **表达式** 的数目。

示例 1：

输入： nums = [1,1,1,1,1], target = 3
输出： 5
解释： 一共有 5 种方法让最终目标和为 3 。
-1 + 1 + 1 + 1 + 1 = 3
+1 - 1 + 1 + 1 + 1 = 3
+1 + 1 - 1 + 1 + 1 = 3
+1 + 1 + 1 - 1 + 1 = 3
+1 + 1 + 1 + 1 - 1 = 3

代码块

```java
class Solution {
    int cnt = 0;
    public int findTargetSumWays(int[] nums, int target) {
        backtrack(nums,target,0);
        return cnt;
    }
    private void backtrack(int[] nums, int target, int index){
        if(index == nums.length){
            if(target == 0) cnt++;
            return;
        }
        backtrack(nums, target-nums[index], index+1);
        backtrack(nums, target+nums[index], index+1);
    }
}
```

LeetCode17. 电话号码的字母组合

> 给定一个仅包含数字 `2-9` 的字符串，返回所有它能表示的字母组合。答案可以按 **任意顺序** 返回。
>
> 给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。



示例 1：

输入： digits = "23"
输出： ["ad","ae","af","bd","be","bf","cd","ce","cf"]

代码块

```java
class Solution {
    String[] map = {"","","abc","def","ghi","jkl","mno","pqrs","tuv","wxyz"};
    public List<String> letterCombinations(String digits) {
        List<String> res = new ArrayList<>();
        backtrack(digits,res,0,new StringBuilder());
        return res;
    }
    private void backtrack(String digits, List<String> res, int index,
StringBuilder path){
        if(index == digits.length()){
            res.add(path.toString());
            return;
        }
        int num = digits.charAt(index) - '0';
        String letters = map[num];
        for(char c : letters.toCharArray()){
            path.append(c);
            backtrack(digits,res,index+1,path);
            path.deleteCharAt(path.length()-1);
        }
    }
}
```

LeetCode78. 子集

> 给你一个整数数组 `nums` ，数组中的元素 **互不相同** 。返回该数组所有可能的子集（幂集）。
>
> 解集 **不能** 包含重复的子集。你可以按 **任意顺序** 返回解集。

示例 1：

输入： nums = [1,2,3]
输出： [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

代码块

```java
class Solution {
    List<List<Integer>> res = new ArrayList<>();
    List<Integer> path = new ArrayList<>();
    public List<List<Integer>> subsets(int[] nums) {
        backtrack(nums,0);
        return res;
    }
    private void backtrack(int[] nums,int cur){
        if(cur == nums.length){
            res.add(new ArrayList<>(path));
```

```
            return;
        }
        path.add(nums[cur]);
        backtrack(nums,cur+1);
        path.remove(path.size()-1);
        backtrack(nums,cur+1);
    }
}
```

LeetCode93. 复原IP地址【华为】

> **有效 IP 地址** 正好由四个整数（每个整数位于 `0` 到 `255` 之间组成，且不能含有前导 `0`），整数之间用 `'.'` 分隔。
>
> - 例如：`"0.1.2.201"` 和 `"192.168.1.1"` 是 **有效** IP 地址，但是 `"0.011.255.245"`、`"192.168.1.312"` 和 `"192.168@1.1"` 是 **无效** IP 地址。
>
> 给定一个只包含数字的字符串 `s` ，用以表示一个 IP 地址，返回所有可能的**有效 IP 地址**，这些地址可以通过在 `s` 中插入 `'.'` 来形成。你 **不能** 重新排序或删除 `s` 中的任何数字。你可以按 **任何** 顺序返回答案。

示例 1：

输入：s = "25525511135"
输出：["255.255.11.135","255.255.111.35"]

核心思路：经典的分割类回溯题

代码块

```
class Solution {

    public List<String> restoreIpAddresses(String s) {
        List<String> res = new ArrayList<>();
        int n = s.length();
        if(n < 4 || n> 12) return res;

        f(res, 0, new ArrayList<>(), s);

        return res;
    }

    public void f(List<String> res, int begin, List<String> curr, String s){
        if(curr.size() == 4 && begin == s.length()){
            res.add(String.join(".", curr));
            return ;
```

```
17          }
18          if(curr.size() > 4 || begin > s.length()){
19              return ;
20          }
21
22          for(int i=1; i<=3; i++){
23              int end = begin+i;
24              if(end > s.length()) break;
25              String segment = s.substring(begin, end);
26              if(segment.length() > 1 && segment.startsWith("0")) continue;
27              if(Integer.parseInt(segment) > 255) continue;
28              curr.add(segment);
29              f(res, end, curr, s);
30              curr.remove(curr.size()-1);
31          }
32      }
33  }
```

LeetCode131. 分割回文串

> 给你一个字符串 `s` ，请你将 `s` 分割成一些 子串，使每个子串都是 回文串 。返回 `s` 所有可能的分割方案。

示例 1：

输入： s = "aab"
输出： [["a","a","b"],["aa","b"]]

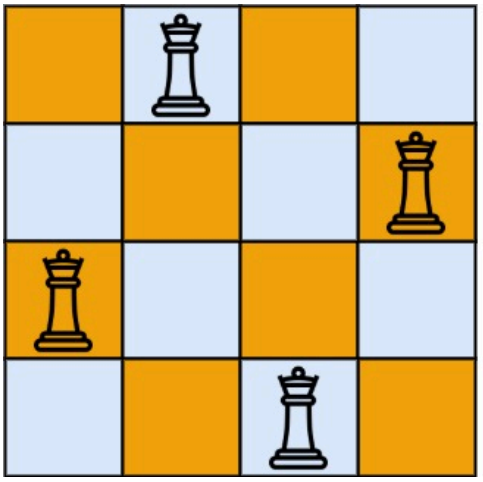
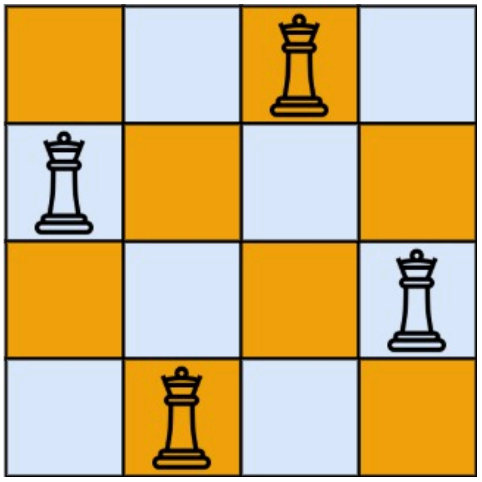
代码块

```
1
```

LeetCode51. N皇后

> 按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。
>
> **n 皇后问题** 研究的是如何将 `n` 个皇后放置在 `n×n` 的棋盘上，并且使皇后彼此之间不能相互攻击。
>
> 给你一个整数 `n` ，返回所有不同的 **n 皇后问题** 的解决方案。
>
> 每一种解法包含一个不同的 **n 皇后问题** 的棋子放置方案，该方案中 `'Q'` 和 `'.'` 分别代表了皇后和空位。

示例 1：

输入： n = 4
输出： [[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]
解释： 如上图所示，4 皇后问题存在两个不同的解法。

代码块

```
1
2
```

---

8. 栈&单调栈

LeetCode20. 有效的括号 【华为】

> 给定一个只包括 `'('`，`')'`，`'{'`，`'}'`，`'['`，`']'` 的字符串 `s` ，判断字符串是否有效。
>
> 有效字符串需满足：
>
> 1. 左括号必须用相同类型的右括号闭合。
> 2. 左括号必须以正确的顺序闭合。
> 3. 每个右括号都有一个对应的相同类型的左括号。

示例 1：

输入： s = "()[]{}"

输出： true

代码块

```java
class Solution {
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for(Character c : s.toCharArray()){
            if(c == '[' || c == '{' || c == '('){
                stack.push(c);
            } else {
                if(stack.isEmpty()) return false;
                char top = stack.pop();
                if(c == ']' && top != '[' || c == '}' && top != '{' || c == ')' && top != '(')
                    return false;
            }
        }
        return stack.isEmpty();
    }
}
```

LeetCode155. 最小栈【华为】

> 设计一个支持 `push` ， `pop` ， `top` 操作，并能在常数时间内检索到最小元素的栈。
>
> 实现 `MinStack` 类:
>
> - `MinStack()` 初始化堆栈对象。
> - `void push(int val)` 将元素val推入堆栈。
> - `void pop()` 删除堆栈顶部的元素。
> - `int top()` 获取堆栈顶部的元素。
> - `int getMin()` 获取堆栈中的最小元素。

代码块

```java
class MinStack {
    Stack<Integer> stack;
    Stack<Integer> minStack;

    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public void push(int val) {
```

```
11          stack.push(val);
12          if(minStack.isEmpty() || val <= minStack.peek()){
13              minStack.push(val);
14          }
15      }
16
17      public void pop() {
18          int top =  stack.pop();
19          if(top == minStack.peek()) minStack.pop();
20      }
21
22      public int top() {
23          return stack.peek();
24      }
25
26      public int getMin() {
27          return minStack.peek();
28      }
29  }
30
```

LeetCode32. 最长有效括号【华为】

> 给你一个只包含 `'('` 和 `')'` 的字符串，找出最长有效（格式正确且连续）括号 子串 的长度。
>
> 左右括号匹配，即每个左括号都有对应的右括号将其闭合的字符串是格式正确的，比如 `"(()())"`。

示例 1：

输入： s = "(()"
输出： 2
解释： 最长有效括号子串是 "()"

代码块

```
1   class Solution {
2       public int longestValidParentheses(String s) {
3           Stack<Integer> stack = new Stack<>();
4           stack.push(-1);
5           int maxlen = 0;
6           for(int i=0; i<s.length(); i++){
7               if(s.charAt(i) == '(') stack.push(i);
8               else {
9                   stack.pop();
```

```
                if(stack.isEmpty()){
                    stack.push(i);
                } else {
                    maxlen = Math.max(maxlen,i-stack.peek());
                }
            }
        }
        return maxlen;
    }
}
```

LeetCode394. 字符串解码【华为】

> 给定一个经过编码的字符串，返回它解码后的字符串。
>
> 编码规则为: `k[encoded_string]`，表示其中方括号内部的 `encoded_string` 正好重复 `k` 次。注意 `k` 保证为正整数。
>
> 你可以认为输入字符串总是有效的；输入字符串中没有额外的空格，且输入的方括号总是符合格式要求的。
>
> 此外，你可以认为原始数据不包含数字，所有的数字只表示重复的次数 `k` ，例如不会出现像 `3a` 或 `2[4]` 的输入。
>
> 测试用例保证输出的长度不会超过 `10(5)`。

示例 1：

输入：s = "3[a2[c]]"
输出："accaccacc"

解题思路：双栈经典解法

1. 准备两个栈：
   - `numStack`：存重复次数
   - `strStack`：存括号前已拼接的字符串
2. 两个临时变量：
   - `curNum`：记录当前累积数字（支持多位数）
   - `curStr`：记录当前正在拼接的字符串
3. 遍历字符分四种情况：
   - **数字**：累积计算多位数
   - **字母**：直接拼到 `curStr`

- **左括号** `[`：把当前数字、当前字符串压栈，重置临时变量
- **右括号** `]`：弹出次数、弹出前缀字符串，把当前字符串重复次数次，拼到前缀上

代码块

```
class Solution {
    public String decodeString(String s) {
        Stack<Integer> numStack = new Stack<>();
        Stack<String> strStack = new Stack<>();
        int curNum = 0;
        StringBuilder curStr = new StringBuilder();
        for(char c : s.toCharArray()){
            // 1. 数字：处理多位数
            if(Character.isDigit(c)){
                curNum = curNum*10+(c-'0');
            } // 2. 左括号：入栈，重置
            else if(c == '['){
                numStack.push(curNum);
                strStack.push(curStr.toString());
                curNum = 0;
                curStr = new StringBuilder();
            } // 3. 右括号：出栈、重复拼接
            else if (c == ']') {
                int k = numStack.pop();
                String preStr = strStack.pop();
                // 把当前字符串重复 k 次
                StringBuilder temp = new StringBuilder(preStr);
                for (int i = 0; i < k; i++) {
                    temp.append(curStr);
                }
                curStr = temp;
            } else { // 4. 普通字符
                curStr.append(c);
            }

        }
        return curStr.toString();
    }
}
```

LeetCode739. 每日温度【华为】

> 给定一个整数数组 `temperatures` ，表示每天的温度，返回一个数组 `answer` ，其中 `answer[i]` 是指对于第 `i` 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，

> 请在该位置用 `0` 来代替。

示例 1:

输入: temperatures = [73,74,75,71,69,72,76,73]
输出: [1,1,4,2,1,1,0,0]

核心思路：单调递减栈

经典单调栈模板题：

- 栈中只存**数组下标**，保持栈对应温度**从底到顶递减**
- 遍历每一天下标 `i`：
  a. 只要栈不为空 **且 当前温度 > 栈顶下标对应的温度**：
     - 弹出栈顶下标 `top`
     - `ans[top] = i - top` 算出间隔天数
  b. 把当前下标 `i` 压入栈

代码块

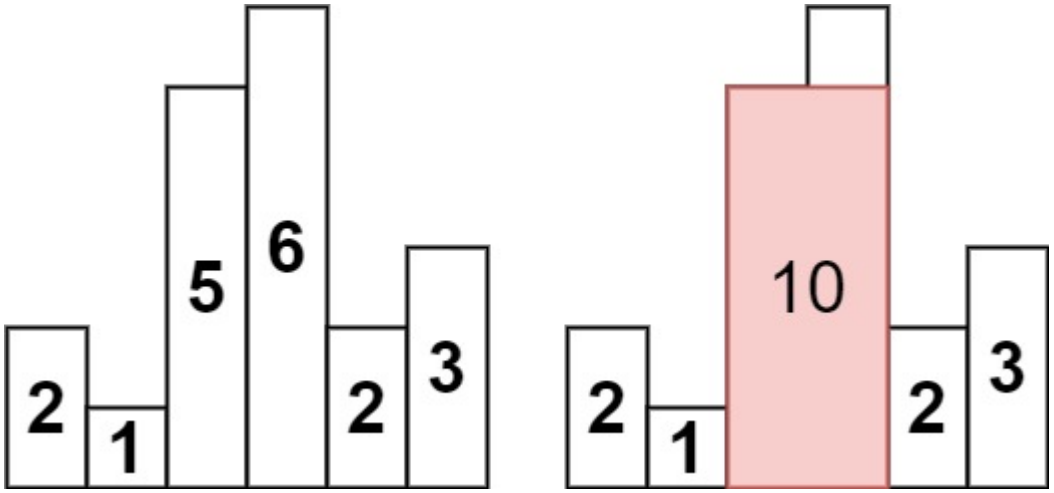
```
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] res = new int[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < n; i++) {
            // 单调递减栈：遇到更大值，弹出并计算天数
            while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
                int top = stack.pop();
                res[top] = i - top;
            }
            stack.push(i);
        }
        return res;
    }
}
```

LeetCode84. 柱状图中的最大矩形

> 给定 *n* 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1 。
>
> 求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例 1:



输入： heights = [2,1,5,6,2,3]
输出： 10
解释： 最大的矩形为图中红色区域，面积为 10

思路解析：单调递增栈

代码块

```
class Solution {
    public int largestRectangleArea(int[] heights) {
        int[] h = new int[heights.length+2];
        System.arraycopy(heights,0,h,1,heights.length);
        Stack<Integer> stack = new Stack<>();
        int maxArea = 0;
        for(int i=0; i<h.length; i++){
            // 维持单调递增，遇到更小则弹出计算
            while(!stack.isEmpty() && h[i] < h[stack.peek()]){
                int height = h[stack.pop()];
                int width = i-stack.peek()-1;
                maxArea = Math.max(maxArea, height*width);
            }
            stack.push(i);
        }
        return maxArea;
    }
}
```

LeetCode23. 合并K个升序链表【华为】

> 给你一个链表数组，每个链表都已经按升序排列。

> 请你将所有链表合并到一个升序链表中，返回合并后的链表。

示例 1：

输入：lists = [[1,4,5],[1,3,4],[2,6]]
输出：[1,1,2,3,4,4,5,6]
解释：链表数组如下：
[
 1->4->5,
 1->3->4,
 2->6
]
将它们合并到一个有序链表中得到。
1->1->2->3->4->4->5->6

核心思路

利用**小顶堆**每次快速获取 k 个链表中当前最小的节点：

1. 初始化小顶堆，将所有非空链表的头节点加入堆中
2. 创建哑节点（dummy）作为结果链表的头，方便构建
3. 循环取出堆顶的最小节点，加入结果链表
4. 如果该节点还有下一个节点，将下一个节点加入堆中
5. 直到堆为空，返回哑节点的下一个节点

代码块

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        // 边界处理：空数组直接返回null
        if (lists == null || lists.length == 0) {
            return null;
        }
        // 初始化小顶堆，自定义比较器按节点值升序排列
        PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) -> a.val - b.val);
        // 将所有非空链表的头节点加入堆
        for (ListNode head : lists) {
            if (head != null) {
                minHeap.offer(head);
            }
        }
        ListNode dummy = new ListNode(-1);
        ListNode curr = dummy;

        // 循环取出最小节点
```

```
19          while (!minHeap.isEmpty()) {
20              ListNode minNode = minHeap.poll();
21              curr.next = minNode;
22              curr = curr.next;
23              // 如果该节点还有下一个节点，加入堆中
24              if (minNode.next != null) {
25                  minHeap.offer(minNode.next);
26              }
27          }
28          return dummy.next;
29      }
30  }
```

---

9. 贪心

LeetCode56. 合并区间

> 以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [start(i), end(i)]` 。请你合并所有重叠的区间，并返回 *一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间*。

示例 1：

输入： intervals = [[1,3],[2,6],[8,10],[15,18]]
输出： [[1,6],[8,10],[15,18]]
解释： 区间 [1,3] 和 [2,6] 重叠, 将它们合并为 [1,6].

解法思路：先排序后比较

代码块

```
1   class Solution {
2       public int[][] merge(int[][] intervals) {
3           Arrays.sort(intervals, new Comparator<>(){
4               public int compare(int[] interval1, int[] interval2){
5                   return interval1[0]-interval2[0];
6               }
7           });
8
9           List<int[]> merge = new ArrayList<>();
10          for(int i=0;i<intervals.length;i++){
```

```
            int l = intervals[i][0]; int r = intervals[i][1];
            if(merge.isEmpty() || merge.get(merge.size()-1)[1] < l){
                merge.add(new int[]{l,r});
            } else {
                merge.get(merge.size()-1)[1] =
    Math.max(merge.get(merge.size()-1)[1], r);
            }
        }
        return merge.toArray(new int[merge.size()][]);
    }
}
```

LeetCode122. 买卖股票的最佳时机II

> 给你一个整数数组 `prices` ，其中 `prices[i]` 表示某支股票第 `i` 天的价格。
>
> 在每一天，你可以决定是否购买和/或出售股票。你在任何时候 **最多** 只能持有 **一股** 股票。然而，你可以在 **同一天** 多次买卖该股票，但要确保你持有的股票不超过一股。
>
> 返回 *你能获得的* ***最大*** *利润* 。

示例 1：

输入： prices = [7,1,5,3,6,4]
输出： 7
解释： 在第 2 天（股票价格 = 1）的时候买入，在第 3 天（股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5 - 1 = 4。
随后，在第 4 天（股票价格 = 3）的时候买入，在第 5 天（股票价格 = 6）的时候卖出, 这笔交易所能获得利润 = 6 - 3 = 3。
最大总利润为 4 + 3 = 7 。

代码块

```
class Solution {
    public int maxProfit(int[] prices) {
        int profit = 0;
        for(int i=1;i<prices.length;i++){
            if(prices[i]>prices[i-1]){
                profit += prices[i]-prices[i-1];
            }
        }
        return profit;
    }
}
```

LeetCode55. 跳跃游戏【华为】

> 给你一个非负整数数组 `nums` ，你最初位于数组的 **第一个下标** 。数组中的每个元素代表你在该位置可以跳跃的最大长度。
>
> 判断你是否能够到达最后一个下标，如果可以，返回 `true` ；否则，返回 `false` 。

示例 1：

输入： nums = [2,3,1,1,4]
输出： true
解释： 可以先跳 1 步，从下标 0 到达下标 1, 然后再从下标 1 跳 3 步到达最后一个下标。

代码块

```
class Solution {
    public boolean canJump(int[] nums) {
        int maxReach = 0;
        for(int i=0; i<nums.length;i++){
            if(i>maxReach) return false;
            maxReach = Math.max(maxReach,i+nums[i]);
            if(maxReach>=nums.length-1) return true;
        }
        return true;
    }
}
```

LeetCode45. 跳跃游戏II【华为】

> 给定一个长度为 `n` 的 **0 索引**整数数组 `nums` 。初始位置在下标 0。
>
> 每个元素 `nums[i]` 表示从索引 `i` 向后跳转的最大长度。换句话说，如果你在索引 `i` 处，你可以跳转到任意 `(i + j)` 处：
>
> - `0 <= j <= nums[i]` 且
> - `i + j < n`
>
> 返回到达 `n - 1` 的最小跳跃次数。测试用例保证可以到达 `n - 1` 。

示例 1:

代码块

```
输入: nums = [2,3,1,1,4]
输出: 2
```

解释： 跳到最后一个位置的最小跳跃数是 2。
从下标为 0 跳到下标为 1 的位置，跳 1 步，然后跳 3 步到达数组的最后一个位置。

代码块

```
class Solution {
    public int jump(int[] nums) {
        int n = nums.length;
        if(n<=1) return 0;
        // steps : 跳跃次数
        int steps = 0;
        // currentEnd : 当前这一步能达到的最远距离
        int currentEnd = 0;
        int maxReach = 0;
        for(int i=0; i<n-1; i++){
            maxReach = Math.max(maxReach, i+nums[i]);
            if(i == currentEnd){
                steps++;
                currentEnd = maxReach;
            }
        }
        return steps;
    }
}
```

LeetCode406. 根据身高重建队列【华为】

> 假设有打乱顺序的一群人站成一个队列，数组 `people` 表示队列中一些人的属性（不一定按顺序）。每个 `people[i] = [h(i), k(i)]` 表示第 `i` 个人的身高为 `h(i)` ，前面 **正好** 有 `k(i)` ()个身高大于或等于 `h(i)` 的人。
>
> 请你重新构造并返回输入数组 `people` 所表示的队列。返回的队列应该格式化为数组 `queue` ，其中 `queue[j] = [h(j), k(j)]` 是队列中第 `j` 个人的属性（`queue[0]` 是排在队列前面的人）。

示例 1：

输入： people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]
输出： [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]
解释：
编号为 0 的人身高为 5 ，没有身高更高或者相同的人排在他前面。
编号为 1 的人身高为 7 ，没有身高更高或者相同的人排在他前面。
编号为 2 的人身高为 5 ，有 2 个身高更高或者相同的人排在他前面，即编号为 0 和 1 的人。

编号为 3 的人身高为 6 ，有 1 个身高更高或者相同的人排在他前面，即编号为 1 的人。
编号为 4 的人身高为 4 ，有 4 个身高更高或者相同的人排在他前面，即编号为 0、1、2、3 的人。
编号为 5 的人身高为 7 ，有 1 个身高更高或者相同的人排在他前面，即编号为 1 的人。
因此 [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] 是重新构造后的队列。

思路解析：

先按**身高降序排序，身高相同则按 k 值升序排序**；然后依次将每个人插入到队列的第 k 个位置。

为什么这个策略有效？

1. **高个子优先**：当我们处理一个人时，所有已经在队列中的人身高都大于或等于他。因此，将他插入到第 k 个位置，正好满足 "前面有 k 个身高大于等于他的人" 的条件。
2. **矮个子不影响高个子**：插入矮个子时，不会改变已经排好的高个子之间的相对顺序，也不会影响高个子的 k 值（因为矮个子的身高小于高个子，不会被计入高个子的 k 值）。
3. **身高相同按 k 升序**：如果两个人身高相同，k 值小的应该排在前面。因为身高相同的人会互相计入对方的 k 值，先插入 k 小的，后插入 k 大的，才能保证两者的 k 值都正确。

代码块

```
class Solution {
    public int[][] reconstructQueue(int[][] people) {
        // 1. 排序：身高降序，身高相同则k值升序
        Arrays.sort(people, (a, b) -> {
            if (a[0] != b[0]) {
                // 身高不同，降序排列
                return b[0] - a[0];
            } else {
                // 身高相同，k值升序排列
                return a[1] - b[1];
            }
        });

        // 2. 依次插入到第k个位置
        List<int[]> queue = new ArrayList<>();
        for (int[] person : people) {
            queue.add(person[1], person);
        }

        // 3. 转换为数组返回
        return queue.toArray(new int[people.length][]);
    }
}
```

LeetCode134. 加油站

> 在一条环路上有 `n` 个加油站，其中第 `i` 个加油站有汽油 `gas[i]` 升。
>
> 你有一辆油箱容量无限的的汽车，从第 `i` 个加油站开往第 `i+1` 个加油站需要消耗汽油 `cost[i]` 升。你从其中的一个加油站出发，开始时油箱为空。
>
> 给定两个整数数组 `gas` 和 `cost` ，如果你可以按顺序绕环路行驶一周，则返回出发时加油站的编号，否则返回 `-1` 。如果存在解，则 **保证** 它是 **唯一** 的。

示例 1:

输入: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
输出: 3
解释:
从 3 号加油站(索引为 3 处)出发，可获得 4 升汽油。此时油箱有 = 0 + 4 = 4 升汽油
开往 4 号加油站，此时油箱有 4 - 1 + 5 = 8 升汽油
开往 0 号加油站，此时油箱有 8 - 2 + 1 = 7 升汽油
开往 1 号加油站，此时油箱有 7 - 3 + 2 = 6 升汽油
开往 2 号加油站，此时油箱有 6 - 4 + 3 = 5 升汽油
开往 3 号加油站，你需要消耗 5 升汽油，正好足够你返回到 3 号加油站。
因此，3 可为起始索引。

代码块

```
class Solution {
    public int canCompleteCircuit(int[] gas, int[] cost) {
        int totalCost = 0;
        int totalGas = 0;
        int currentTank = 0;
        int start = 0;
        for(int i=0; i<gas.length; i++){
            totalCost += cost[i];
            totalGas += gas[i];
            currentTank += gas[i]-cost[i];
            if(currentTank<0){
                currentTank = 0;
                start = i+1;
            }
        }
        return totalGas >= totalCost ? start : -1;
    }
}
```

LeetCode621. 任务调度器

> 给你一个用字符数组 `tasks` 表示的 CPU 需要执行的任务列表，用字母 A 到 Z 表示，以及一个冷却时间 `n` 。每个周期或时间间隔允许完成一项任务。任务可以按任何顺序完成，但有一个限制：两个**相同种类**的任务之间必须有长度为 `n` 的冷却时间。
>
> 返回完成所有任务所需要的**最短时间间隔**。

示例 1：

输入：tasks = ["A","A","A","B","B","B"], n = 2

输出：8

解释：

在完成任务 A 之后，你必须等待两个间隔。对任务 B 来说也是一样。在第 3 个间隔，A 和 B 都不能完成，所以你需要待命。在第 4 个间隔，由于已经经过了 2 个间隔，你可以再次执行 A 任务。

思路解析：计算框架时间：`（最大频次-1）*（n+1）+最大频次次数`

代码块

```
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] cnt = new int[26];
        for(char c : tasks) cnt[c-'A']++;
        int maxFreq = 0;
        for(int num : cnt){
            maxFreq = Math.max(maxFreq,num);
        }
        int maxCount = 0;

        for(int num : cnt){
            if(num == maxFreq) maxCount++;
        }
        int time = (maxFreq-1)*(n+1)+maxCount;
        return Math.max(time,tasks.length);
    }
}
```

10. 哈希表

LeetCode1. 两数之和【华为】

> 给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** `target` 的那 **两个** 整数，并返回它们的数组下标。
>
> 你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。
>
> 你可以按任意顺序返回答案。

示例 1：

输入： nums = [2,7,11,15], target = 9
输出： [0,1]
解释： 因为 nums[0] + nums[1] == 9 ，返回 [0, 1] 。

代码块

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap<Integer,Integer> map = new HashMap<>();
        for(int i=0;i<nums.length;i++){
            int other = target-nums[i];
            if(map.containsKey(other)){
                return new int[]{i,map.get(other)};
            }
            map.put(nums[i],i);
        }
        return new int[0];
    }
}
```

LeetCode49. 字母异位词分组

> 给你一个字符串数组，请你将 **字母异位词** 组合在一起。可以按任意顺序返回结果列表。

示例 1:

输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

输出: [["bat"],["nat","tan"],["ate","eat","tea"]]

解释：

- 在 strs 中没有字符串可以通过重新排列来形成 `"bat"`。
- 字符串 `"nat"` 和 `"tan"` 是字母异位词，因为它们可以重新排列以形成彼此。
- 字符串 `"ate"` ， `"eat"` 和 `"tea"` 是字母异位词，因为它们可以重新排列以形成彼此。

代码块

```java
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        HashMap<String,List<String>> map = new HashMap<>();
        for(String str : strs){
            char[] charArray = str.toCharArray();
            Arrays.sort(charArray);
            String key = new String(charArray);
            if(!map.containsKey(key)){
                map.put(key,new ArrayList<>());
            }
            map.get(key).add(str);
        }
        return new ArrayList<>(map.values());
    }
}
```

LeetCode41. 缺失的第一个正数

> 给你一个未排序的整数数组 `nums` ，请你找出其中没有出现的最小的正整数。
>
> 请你实现时间复杂度为 `O(n)` 并且只使用常数级别额外空间的解决方案。

示例 1：

输入： nums = [1,2,0]
输出： 3
解释： 范围 [1,2] 中的数字都在数组中。

思路解析：原地哈希

代码块

```java
class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for(int i=0; i<n; i++){
            while(nums[i]>=1 && nums[i]<=n && nums[nums[i]-1] != nums[i]){
                int temp = nums[nums[i]-1];
                nums[nums[i]-1] = nums[i];
                nums[i] = temp;
            }
        }
        for(int i=0; i<n; i++){
            if(nums[i] != i+1){
                return i+1;
            }
```

```
            }
            return n+1;
        }
    }
```

LeetCode560. 和为K的子数组【华为】

> 给你一个整数数组 `nums` 和一个整数 `k` ，请你统计并返回 *该数组中和为 `k` 的子数组的个数*。
>
> 子数组是数组中元素的连续非空序列。

示例 1：

输入：nums = [1,1,1], k = 2
输出：2

思路解析：前缀和+哈希表

代码块

```java
class Solution {
    public int subarraySum(int[] nums, int k) {
        // key : 前缀和      value : 出现次数
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0,1);
        // preSum : 前缀和
        int preSum = 0;
        int count = 0;
        for(int num : nums){
            preSum += num;
            // 查找有多少个前缀和 = preSum-k
            if(map.containsKey(preSum-k)){
                count += map.get(preSum-k);
            }
            map.put(preSum, map.getOrDefault(preSum,0)+1);
        }
        return count;
    }
}
```

LeetCode347. 前k个高频元素【华为】

> 给你一个整数数组 `nums` 和一个整数 `k` ，请你返回其中出现频率前 `k` 高的元素。你可以按 **任意顺序** 返回答案。

示例 1：

输入： nums = [1,1,1,2,2,3], k = 2

输出： [1,2]

代码块

```java
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        // 1. 统计频率
        Map<Integer, Integer> countMap = new HashMap<>();
        for (int num : nums) {
            countMap.put(num, countMap.getOrDefault(num, 0) + 1);
        }

        // 2. 小根堆：按频率升序
        PriorityQueue<Map.Entry<Integer, Integer>> minHeap =
            new PriorityQueue<>((a, b) -> a.getValue() - b.getValue());

        // 3. 遍历入堆，维持堆大小为k
        for (Map.Entry<Integer, Integer> entry : countMap.entrySet()) {
            minHeap.offer(entry);
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }

        // 4. 取出结果
        int[] res = new int[k];
        int idx = 0;
        while (!minHeap.isEmpty()) {
            res[idx++] = minHeap.poll().getKey();
        }
        return res;
    }
}
```

11. 图&DFS&BFS

LeetCode200. 岛屿数量【华为】

> 给你一个由 `'1'` （陆地）和 `'0'` （水）组成的的二维网格，请你计算网格中岛屿的数量。
>
> 岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。
>
> 此外，你可以假设该网格的四条边均被水包围。

示例 1：

输入： grid = [
['1','1','1','1','0'],
['1','1','0','1','0'],
['1','1','0','0','0'],
['0','0','0','0','0']
]
输出： 1

DFS

```
class Solution {
    public int numIslands(char[][] grid) {
        int row = grid.length;
        int col = grid[0].length;
        int count = 0;
        for(int i=0;i<row;i++){
            for(int j=0;j<col;j++){
                if(grid[i][j] == '1'){
                    count++;
                    dfs(grid,i,j);
                }
            }
        }
        return count;
    }
    public void dfs(char[][] grid, int i, int j){
        if(i<0 || i>=grid.length || j<0 || j>=grid[0].length || grid[i][j] == '0') return;
        grid[i][j] = '0';
        dfs(grid,i+1,j);
        dfs(grid,i-1,j);
        dfs(grid,i,j+1);
        dfs(grid,i,j-1);
    }
}
```

BFS

```
class Solution {
```

```
    private static int[][] dir = {{0,1},{0,-1},{1,0},{-1,0}};
    public int numIslands(char[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int cnt = 0;
        for(int i=0; i<m; i++){
            for(int j=0; j<n; j++){
                if(grid[i][j] == '1'){
                    cnt++;
                    bfs(grid,i,j);
                }
            }
        }
        return cnt;
    }

    private void bfs(char[][] grid, int i, int j){
        // 队列存储待访问的陆地坐标（用int数组存储行和列）
        Queue<int[]> queue = new LinkedList<>();
        queue.offer(new int[]{i,j});
        grid[i][j] = '0';

        while(!queue.isEmpty()){
            int[] curr = queue.poll();
            int x = curr[0];
            int y = curr[1];

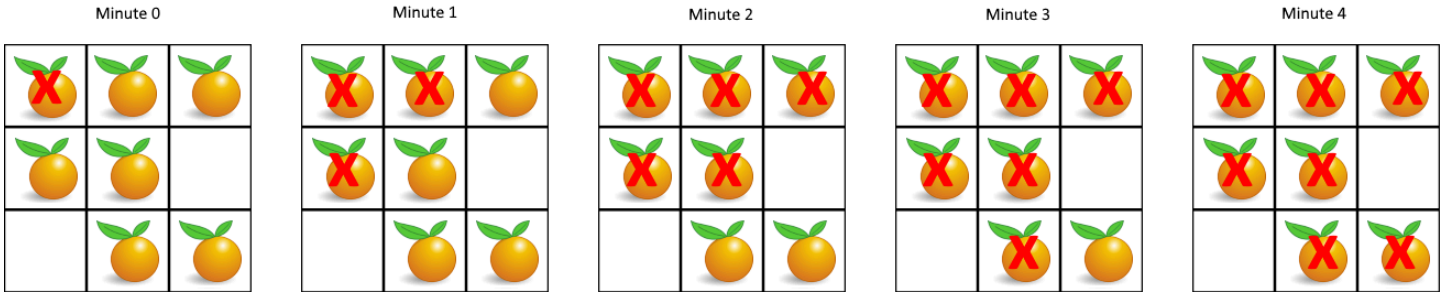
            for(int[] d : dir){
                int a = x + d[0];
                int b = y + d[1];

                if(a>=0 && a<grid.length && b>=0 && b<grid[0].length && grid[a][b] == '1'){
                    // 加入队列并标记为已访问（关键：入队时标记，而非出队时）
                    queue.offer(new int[]{a,b});
                    grid[a][b] = '0';
                }
            }
        }
    }
}
```

LeetCode994. 腐烂的橘子

> 在给定的 `m x n` 网格 `grid` 中，每个单元格可以有以下三个值之一：
>
> - 值 `0` 代表空单元格；
> - 值 `1` 代表新鲜橘子；
> - 值 `2` 代表腐烂的橘子。
>
> 每分钟，腐烂的橘子 **周围 4 个方向上相邻** 的新鲜橘子都会腐烂。
>
> 返回 *直到单元格中没有新鲜橘子为止所必须经过的最小分钟数。如果不可能，返回* `-1` 。

示例 1：



输入： grid = [[2,1,1],[1,1,0],[0,1,1]]
输出： 4

```
代码块

1
```

LeetCode208. 实现Trie（前缀树）

> **Trie**（发音类似 "try"）或者说 **前缀树** 是一种树形数据结构，用于高效地存储和检索字符串数据集中的键。这一数据结构有相当多的应用情景，例如自动补全和拼写检查。
>
> 请你实现 Trie 类：
>
> - `Trie()` 初始化前缀树对象。
> - `void insert(String word)` 向前缀树中插入字符串 `word` 。
> - `boolean search(String word)` 如果字符串 `word` 在前缀树中，返回 `true` （即，在检索之前已经插入）；否则，返回 `false` 。
> - `boolean startsWith(String prefix)` 如果之前已经插入的字符串 `word` 的前缀之一为 `prefix` ，返回 `true` ；否则，返回 `false` 。

示例：

输入
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]

[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
输出
[null, null, true, false, true, null, true]

解释
Trie trie = new Trie();
trie.insert("apple");
trie.search("apple"); // 返回 True
trie.search("app"); // 返回 False
trie.startsWith("app"); // 返回 True
trie.insert("app");
trie.search("app"); // 返回 True

```
代码块
1
```

LeetCode207. 课程表【华为】

> 你这个学期必须选修 `numCourses` 门课程，记为 `0` 到 `numCourses - 1` 。
>
> 在选修某些课程之前需要一些先修课程。 先修课程按数组 `prerequisites` 给出，其中 `prerequisites[i] = [a(i), b(i)]` ，表示如果要学习课程 `a(i)` 则 **必须** 先学习课程 `b(i)` ()。
>
> - 例如，先修课程对 `[0, 1]` 表示：想要学习课程 `0` ，你需要先完成课程 `1` 。
>
> 请你判断是否可能完成所有课程的学习？如果可以，返回 `true` ；否则，返回 `false` 。、

示例 1：

输入：numCourses = 2, prerequisites = [[1,0]]
输出：true
解释：总共有 2 门课程。学习课程 1 之前，你需要完成课程 0 。这是可能的。

思路解析：

拓扑排序

判断**有向图中是否存在环**。如果存在环，说明有课程的先修关系形成了死循环，无法完成所有课程；如果不存在环（即图是有向无环图 DAG），则可以完成。

BFS:

构建图的**邻接表**和**入度数组**

将所有入度为 0 的节点（没有先修课的课程）加入队列

依次取出队列中的节点，遍历它的所有后继节点，将后继节点的入度减 1

如果某个后继节点的入度变为 0，说明它的所有先修课都已完成，加入队列

最后统计已访问的节点数：

- 若等于总课程数 → 无环，返回 `true`
- 若小于总课程数 → 有环，返回 `false`

BFS:基于入度的概念，通过逐步移除入度为 0 的节点来检测环：

```java
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        // 1. 构建邻接表和入度数组
        ArrayList<Integer>[] adjacency = new ArrayList[numCourses];
        int[] inDegree = new int[numCourses];

        // 初始化邻接表
        for (int i = 0; i < numCourses; i++) {
            adjacency[i] = new ArrayList<>();
        }

        // 填充邻接表和入度数组
        for (int[] pre : prerequisites) {
            int course = pre[0];    // 要学的课程
            int prerequisite = pre[1]; // 先修课程
            adjacency[prerequisite].add(course); // 边: prerequisite → course
            inDegree[course]++;
        }

        // 2. 初始化队列，加入所有入度为0的节点
        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) {
            if (inDegree[i] == 0) {
                queue.offer(i);
            }
        }

        // 3. BFS遍历
        int visitedCount = 0;
        while (!queue.isEmpty()) {
            int curr = queue.poll();
            visitedCount++;

```

```
            // 遍历当前节点的所有后继节点
            for (int next : adjacency[curr]) {
                inDegree[next]--;
                // 入度变为0，说明所有先修课已完成
                if (inDegree[next] == 0) {
                    queue.offer(next);
                }
            }
        }

        // 4. 判断是否所有课程都被访问过
        return visitedCount == numCourses;
    }
}
```

DFS:

通过递归遍历每个节点，用**状态数组**记录节点的访问状态，检测是否存在环：

- 状态 `0`：未访问
- 状态 `1`：正在访问（在当前递归栈中）
- 状态 `2`：已访问完成

DFS

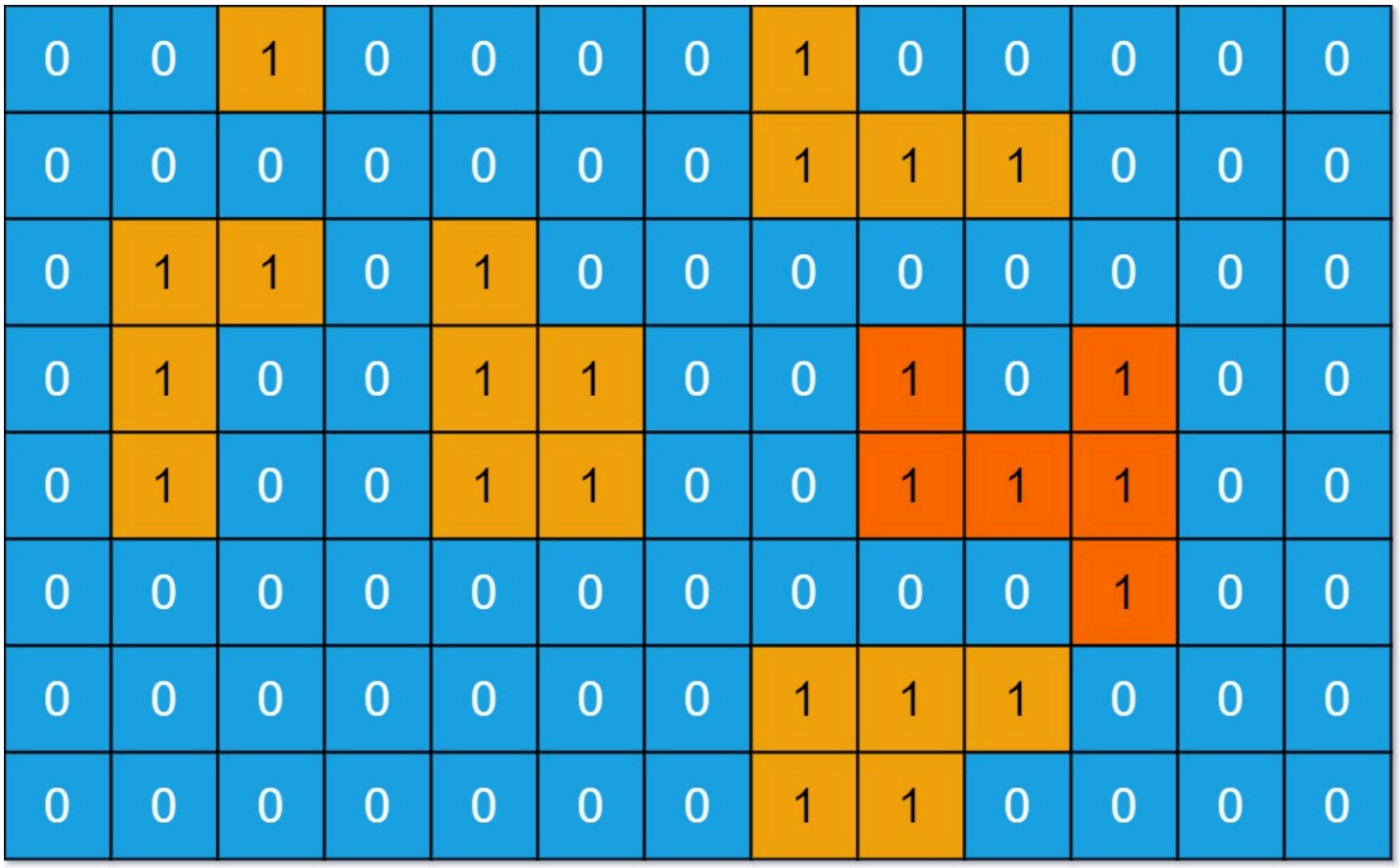
```
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        // 1. 构建邻接表
        ArrayList<Integer>[] adjacency = new ArrayList[numCourses];
        for (int i = 0; i < numCourses; i++) {
            adjacency[i] = new ArrayList<>();
        }
        for (int[] pre : prerequisites) {
            adjacency[pre[1]].add(pre[0]);
        }
        // 2. 状态数组：0=未访问，1=正在访问，2=已访问
        int[] status = new int[numCourses];
        // 3. 遍历所有节点
        for (int i = 0; i < numCourses; i++) {
            if (hasCycle(i, adjacency, status)) {
                return false;
            }
        }
        return true;
    }
    // 检测以curr为起点的路径是否存在环
```

```java
    private boolean hasCycle(int curr, ArrayList<Integer>[] adjacency, int[] status) {
        // 如果当前节点正在访问，说明存在环
        if (status[curr] == 1) {
            return true;
        }
        // 如果当前节点已访问完成，说明无环
        if (status[curr] == 2) {
            return false;
        }
        // 标记为正在访问
        status[curr] = 1;
        // 递归遍历所有后继节点
        for (int next : adjacency[curr]) {
            if (hasCycle(next, adjacency, status)) {
                return true;
            }
        }
        // 所有后继节点遍历完成，标记为已访问
        status[curr] = 2;
        return false;
    }
}
```

LeetCode695. 岛屿的最大面积【华为】

> 给你一个大小为 `m x n` 的二进制矩阵 `grid` 。
>
> **岛屿** 是由一些相邻的 `1` (代表土地) 构成的组合，这里的「相邻」要求两个 `1` 必须在 **水平或者竖直的四个方向上** 相邻。你可以假设 `grid` 的四个边缘都被 `0` （代表水）包围着。
>
> 岛屿的面积是岛上值为 `1` 的单元格的数目。
>
> 计算并返回 `grid` 中最大的岛屿面积。如果没有岛屿，则返回面积为 `0` 。

示例 1：



输入： grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,1,1,0,0,0,0]]

输出： 6

解释： 答案不应该是 11 ，因为岛屿只能包含水平或垂直这四个方向上的 1 。

BFS

```java
class Solution {
    // 四个方向：上、下、左、右
    private static final int[][] DIRECTIONS = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

    public int maxAreaOfIsland(int[][] grid) {
        // 边界处理：空网格直接返回0
        if (grid == null || grid.length == 0 || grid[0].length == 0) {
            return 0;
        }
        int rowCount = grid.length;
        int colCount = grid[0].length;
        int maxArea = 0;

        // 遍历网格每一个单元格
        for (int i = 0; i < rowCount; i++) {
            for (int j = 0; j < colCount; j++) {
                // 发现未访问的陆地，启动BFS计算面积
```

```
                if (grid[i][j] == 1) {
                    int currentArea = bfs(grid, i, j, rowCount, colCount);
                    maxArea = Math.max(maxArea, currentArea);
                }
            }
        }
        return maxArea;
    }
    // BFS计算单个岛屿的面积，并标记所有连通陆地为已访问
    private int bfs(int[][] grid, int startRow, int startCol, int rowCount,
int colCount) {
        Queue<int[]> queue = new LinkedList<>();
        queue.offer(new int[]{startRow, startCol});
        grid[startRow][startCol] = 0; // 入队即标记为已访问

        int area = 0;

        while (!queue.isEmpty()) {
            int[] curr = queue.poll();
            area++; // 每出队一个节点，面积+1

            int currRow = curr[0];
            int currCol = curr[1];

            // 遍历四个方向
            for (int[] dir : DIRECTIONS) {
                int newRow = currRow + dir[0];
                int newCol = currCol + dir[1];

                // 检查新坐标是否合法且是未访问的陆地
                if (newRow >= 0 && newRow < rowCount
                    && newCol >= 0 && newCol < colCount
                    && grid[newRow][newCol] == 1) {
                    queue.offer(new int[]{newRow, newCol});
                    grid[newRow][newCol] = 0; // 入队即标记，避免重复入队
                }
            }
        }

        return area;
    }
}
```

DFS

```
class Solution {
```

```java
    private static final int[][] DIRECTIONS = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
    public int maxAreaOfIsland(int[][] grid) {
        if (grid == null || grid.length == 0 || grid[0].length == 0) {
            return 0;
        }

        int rowCount = grid.length;
        int colCount = grid[0].length;
        int maxArea = 0;

        for (int i = 0; i < rowCount; i++) {
            for (int j = 0; j < colCount; j++) {
                if (grid[i][j] == 1) {
                    int currentArea = dfs(grid, i, j, rowCount, colCount);
                    maxArea = Math.max(maxArea, currentArea);
                }
            }
        }

        return maxArea;
    }
    // DFS递归计算单个岛屿的面积
    private int dfs(int[][] grid, int row, int col, int rowCount, int colCount) {
        // 递归终止条件：坐标越界 或 不是未访问的陆地
        if (row < 0 || row >= rowCount || col < 0 || col >= colCount || grid[row][col] == 0) {
            return 0;
        }

        // 标记为已访问
        grid[row][col] = 0;

        // 当前节点面积1 + 四个方向邻居的面积
        int area = 1;
        for (int[] dir : DIRECTIONS) {
            area += dfs(grid, row + dir[0], col + dir[1], rowCount, colCount);
        }

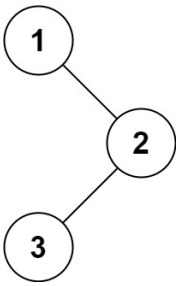
        return area;
    }
}
```

12. 二叉树

LeetCode94. 二叉树的中序遍历

> 给定一个二叉树的根节点 `root` ，返回 *它的* ***中序*** *遍历*。

示例 1：



输入：root = [1,null,2,3]
输出：[1,3,2]

代码块

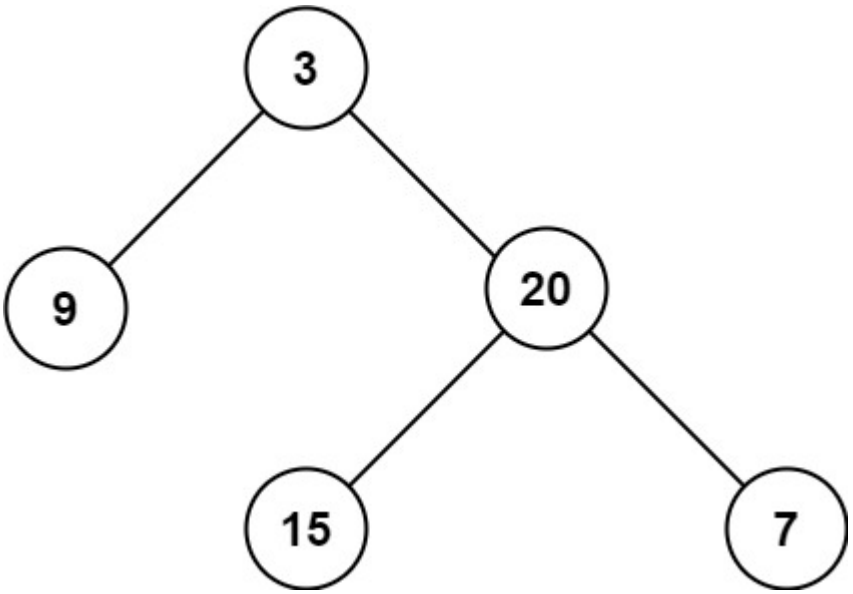
```
class Solution {
    private List<Integer> res = new ArrayList<>();
    public List<Integer> inorderTraversal(TreeNode root) {
        inorder(root);
        return res;
    }
    private void inorder(TreeNode root){
        if(root == null) return;
        inorder(root.left);
        res.add(root.val);
        inorder(root.right);
    }
}
```

LeetCode104. 二叉树的最大深度

> 给定一个二叉树 `root` ，返回其最大深度。

二叉树的 **最大深度** 是指从根节点到最远叶子节点的最长路径上的节点数。

示例 1：



输入：root = [3,9,20,null,null,15,7]
输出：3

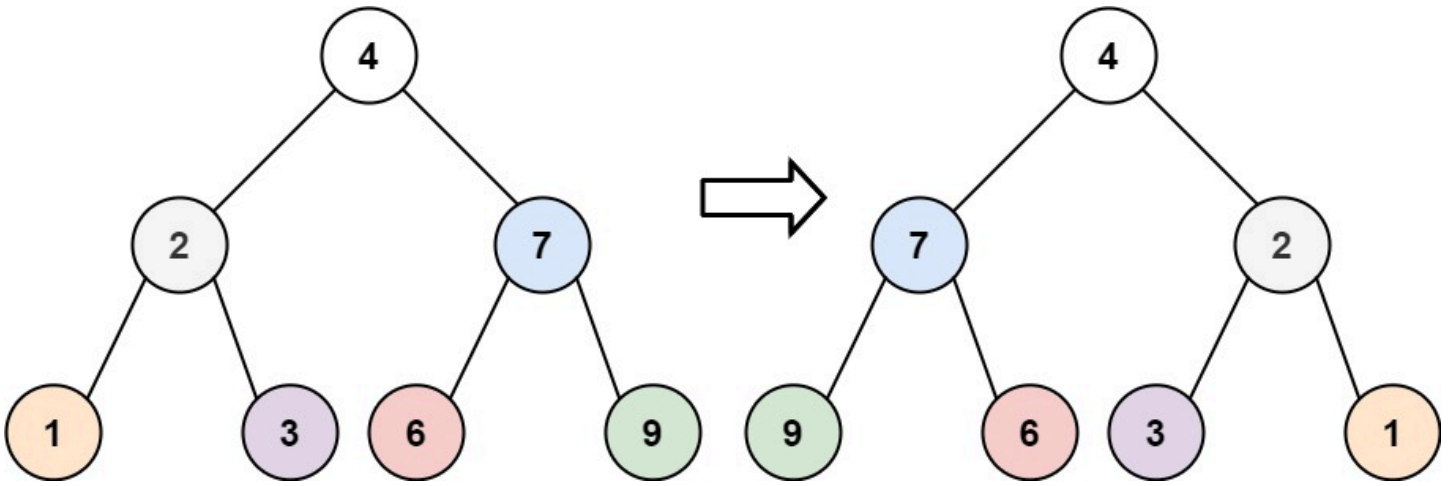
代码块

```
class Solution {
    public int maxDepth(TreeNode root) {
        if(root ==  null) return 0;
        int leftMax = maxDepth(root.left);
        int rightMax = maxDepth(root.right);
        return Math.max(leftMax,rightMax)+1;
    }
}
```

LeetCode226. 翻转二叉树

> 给你一棵二叉树的根节点 `root` ，翻转这棵二叉树，并返回其根节点。

示例 1：



输入： root = [4,2,7,1,3,6,9]
输出： [4,7,2,9,6,3,1]

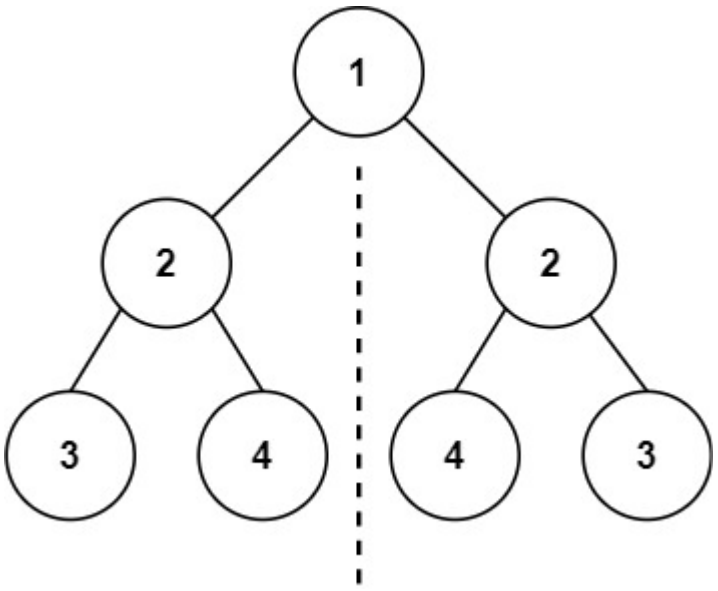
代码块

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if(root == null) return null;
        TreeNode left = invertTree(root.left);
        TreeNode right = invertTree(root.right);
        root.left = right;
        root.right = left;
        return root;
    }
}
```

LeetCode101. 对称二叉树

> 给你一个二叉树的根节点 `root` ， 检查它是否轴对称。

示例 1：



输入： root = [1,2,2,3,4,4,3]
输出： true

代码块

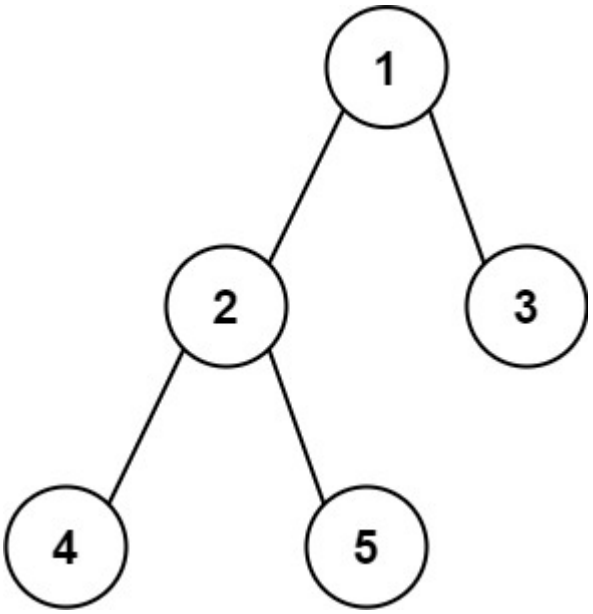
```
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return isMirror(root.left,root.right);
    }
```

```
    public boolean isMirror(TreeNode left, TreeNode right){
        if(left == null && right == null) return true;
        if(left == null || right == null) return false;
        return left.val == right.val && isMirror(left.left,right.right) &&
isMirror(left.right, right.left);
    }
}
```

LeetCode543. 二叉树的直径

> 给你一棵二叉树的根节点，返回该树的 **直径** 。
>
> 二叉树的 **直径** 是指树中任意两个节点之间最长路径的 **长度** 。这条路径可能经过也可能不经过根节点 `root` 。
>
> 两节点之间路径的 **长度** 由它们之间边数表示。

示例 1：



输入： root = [1,2,3,4,5]
输出： 3
解释： 3 ，取路径 [4,2,1,3] 或 [5,2,1,3] 的长度。

代码块

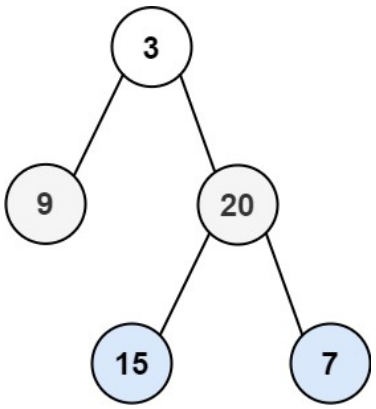
```
class Solution {
    private int max = -1;
    public int diameterOfBinaryTree(TreeNode root) {
        dfs(root);
        return max;
    }
```

```
    private int dfs(TreeNode root){
        if(root == null) return 0;
        int left_deep = dfs(root.left);
        int right_deep = dfs(root.right);
        max = Math.max(left_deep+right_deep,max);
        return Math.max(left_deep,right_deep)+1;
    }
}
```

LeetCode102. 二叉树的层序遍历【华为】

> 给你二叉树的根节点 `root` ，返回其节点值的 **层序遍历** 。 （即逐层地，从左到右访问所有节点）。

示例 1：



输入： root = [3,9,20,null,null,15,7]
输出： [[3],[9,20],[15,7]]

代码块

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        Deque<TreeNode> queue = new LinkedList<>();
        if(root == null) return res;
        queue.offer(root);

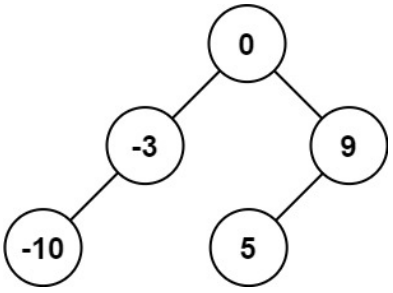
        while(!queue.isEmpty()){
            List<Integer> level = new ArrayList<>();
            int size = queue.size();
            for(int i=0; i<size; i++){
                TreeNode cur = queue.poll();
                level.add(cur.val);
                if(cur.left != null) queue.offer(cur.left);
```

```
                if(cur.right != null) queue.offer(cur.right);
            }
            res.add(level);
        }
        return res;
    }
}
```

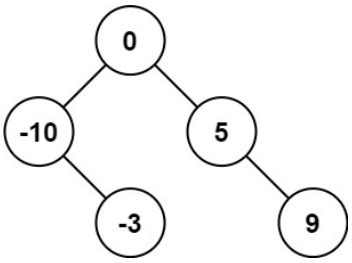
LeetCode108. 将有序数组转换为二叉搜索树

> 给你一个整数数组 `nums` ，其中元素已经按 **升序** 排列，请你将其转换为一棵 平衡 二叉搜索树。

示例 1：



输入： nums = [-10,-3,0,5,9]
输出： [0,-3,9,-10,null,5]
解释： [0,-10,5,null,-3,null,9] 也将被视为正确答案：



代码块

```
class Solution {
     public TreeNode sortedArrayToBST(int[] nums) {
        return buildTree(nums, 0, nums.length - 1);
    }

    // 递归构建[l,r]区间的平衡BST
    private TreeNode buildTree(int[] nums, int l, int r) {
        // 递归终止
        if (l > r) return null;

        // 取中点做根，避免溢出
```

```
            int mid = l + (r - l) / 2;
            TreeNode root = new TreeNode(nums[mid]);

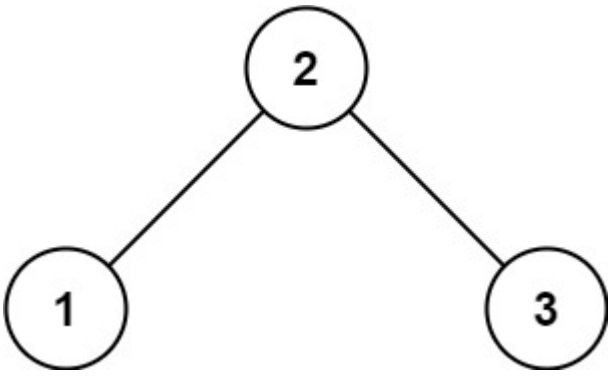
            root.left = buildTree(nums, l, mid - 1);
            root.right = buildTree(nums, mid + 1, r);

            return root;
        }
    }
```

LeetCode98. 验证二叉搜索树

> 给你一个二叉树的根节点 `root` ，判断其是否是一个有效的二叉搜索树。
>
> **有效** 二叉搜索树定义如下：
>
> - 节点的左子树只包含 **严格小于** 当前节点的数。
> - 节点的右子树只包含 **严格大于** 当前节点的数。
> - 所有左子树和右子树自身必须也是二叉搜索树。

示例 1：



输入： root = [2,1,3]
输出： true

代码块

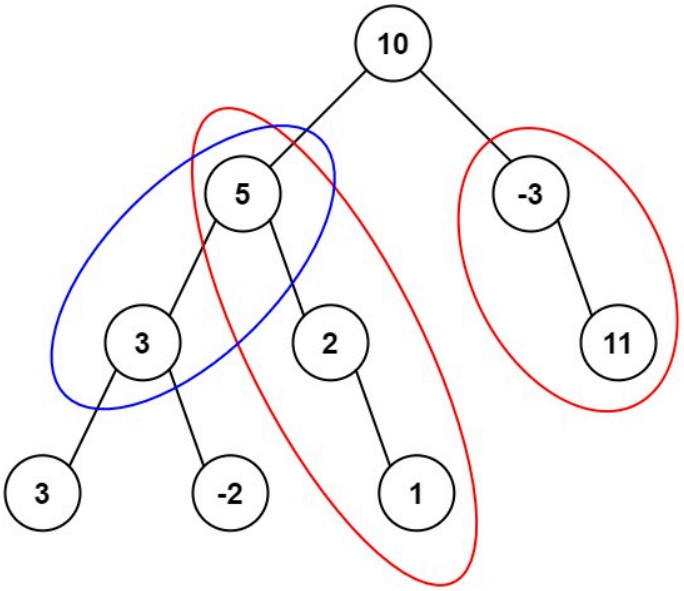
```
class Solution {
    public TreeNode prev = null;
    public boolean isValidBST(TreeNode root) {
        if(root == null) return true;
        if(!isValidBST(root.left)) return false;
        if(prev != null && prev.val >= root.val) return false;
        prev = root;
        return isValidBST(root.right);
    }
```

```
10    }
```

LeetCode437. 路径总和III

> 给定一个二叉树的根节点 `root` ，和一个整数 `targetSum` ，求该二叉树里节点值之和等于 `targetSum` 的 **路径** 的数目。
>
> **路径** 不需要从根节点开始，也不需要在叶子节点结束，但是路径方向必须是向下的（只能从父节点到子节点）。

示例 1：



输入： root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8
输出： 3
解释： 和等于 8 的路径有 3 条，如图所示。

代码块

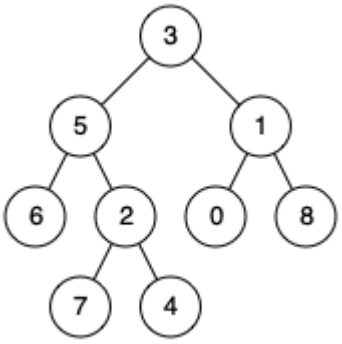
```
1   class Solution {
2       public int pathSum(TreeNode root, int targetSum) {
3           if(root == null) return 0;
4           return
    countPath(root,targetSum)+pathSum(root.left,targetSum)+pathSum(root.right,targe
    tSum);
5       }
6
7       private int countPath(TreeNode node, long targetSum){
8           if(node == null) return 0;
9           int res = 0;
10          if(node.val == targetSum) res++;
```

```
            res += countPath(node.left,targetSum-node.val);
            res += countPath(node.right,targetSum-node.val);
            return res;
        }
    }
```

LeetCode236. 二叉树的最近公共祖先

> 给定一个二叉树, 找到该树中两个指定节点的最近公共祖先。
>
> 百度百科中最近公共祖先的定义为：“对于有根树 T 的两个节点 p、q，最近公共祖先表示为一个节点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（**一个节点也可以是它自己的祖先**）。”

示例 1：



> **输入：** root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
> **输出：** 3
> **解释：** 节点 5 和节点 1 的最近公共祖先是节点 3

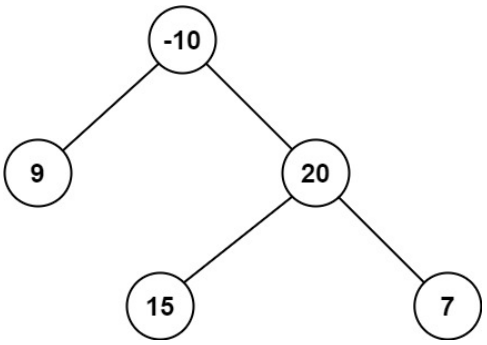
代码块

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode
q) {
        if(root == null || root == p || root == q){
            return root;
        }
        TreeNode left = lowestCommonAncestor(root.left,p,q);
        TreeNode right = lowestCommonAncestor(root.right,p,q);
        if(left!=null && right!=null) return root;
        return left!=null ? left : right;
    }
}
```

LeetCode124. 二叉树的最大路径和【华为】

> 二叉树中的 **路径** 被定义为一条节点序列，序列中每对相邻节点之间都存在一条边。同一个节点在一条路径序列中 **至多出现一次** 。该路径 **至少包含一个** 节点，且不一定经过根节点。
>
> **路径和** 是路径中各节点值的总和。
>
> 给你一个二叉树的根节点 `root` ，返回其 **最大路径和** 。

示例 1：



输入：root = [-10,9,20,null,null,15,7]
输出：42
解释：最优路径是 15 -> 20 -> 7 ，路径和为 15 + 20 + 7 = 42

思路解析：后序遍历，对于每个节点，计算以该节点为最高点的最大路径和

代码块

```java
class Solution {
    private int maxSum;
    public int maxPathSum(TreeNode root) {
        maxSum = Integer.MIN_VALUE;
        dfs(root);
        return maxSum;
    }

    private int dfs(TreeNode root){
        if(root == null) return 0;
        // 左子树最大贡献，负数则不取
        int left = Math.max(dfs(root.left),0);
        int right = Math.max(dfs(root.right),0);

        int current = left + root.val + right;
        maxSum = Math.max(maxSum, current);
        // 向上返回，只能选一条链
        return root.val + Math.max(left,right);
    }
```

```
20   }
```

13. 二分查找